In the Name of God

Bachelor's Thesis in Computer Engineering

Topic

A Unified Approach to Network Design, Purpose-Driven Automation of Operational Scenarios with Ansible, and Monitoring Implementation with Zabbix in a Small Organization

By

Mohammad Hossein Sheikhi

Student No. 140121507

Supervisor

Dr. Parham Arjmand

Academic Year 2025–2026

Table of Contents

Abstract

As computer networks in enterprise environments keep growing in size and complexity, the old way of managing them—mostly hand-typing configurations through the command line (CLI)—just can't keep up anymore with the things that matter most: scalability, agility, and reliability. On top of being slow, manual management is the number one cause of human error, and that leads to unexpected outages across the infrastructure [1]. This thesis sets out to present a comprehensive, unified, and modern approach to infrastructure management by designing, deploying, automating, and monitoring a computer network built around enterprise standards. The project is organized into three main phases. In the first phase, the network was designed in a structured way using hierarchical models, and core services like DHCP and DNS were set up on Windows Server inside the GNS3 emulator. In the second phase, using the Infrastructure as Code (IaC) paradigm and the open-source Ansible platform, device configuration was fully automated without needing any software agent (agentless). Finally, in the third phase, the powerful Zabbix monitoring system was deployed for real-time monitoring of resources, traffic, and events, so the network would stay stable and observable. The results show that combining solid design with automation and centralized monitoring leads to a significant reduction in service deployment time, eliminates configuration errors, and boosts operational efficiency across the network's life cycle.

Chapter One: Introduction and Project Overview

1.1. Introduction

Over the last few decades, computer networks—as the foundational medium for organizational data exchange and service delivery—have expanded at an unprecedented rate, both in physical and in the variety of logical services they carry. The rise of ideas like cloud computing, the Internet of Things (IoT), and big data has put extra pressure on communication infrastructure. In a setting like this, network admins and infrastructure engineers need approaches that can deliver high-availability networks with the lowest possible latency and disruption.

Even though network hardware like routers and switches has come a long way, the way we manage and configure them stayed basically the same for years. The traditional method of network management relies on the admin connecting directly to each individual device through protocols like SSH or Telnet and typing commands in by hand. With the arrival of modern architectures and the need for constant, fast changes (especially in DevOps-based environments), this approach has turned into a serious bottleneck [2]. That's why moving toward software-defined networking (SDN) and network automation has become something you really can't avoid in computer engineering and IT. Recognizing this, the project tries to model the full life cycle of building a network—from scratch (design) to rollout (automation) to upkeep (monitoring)—in a hands-on way that follows current global standards.

1.2. Problem Statement

One of the biggest challenges for enterprise networks today is change management and keeping configurations consistent. Research on service failures consistently identifies human error as one of the primary causes of unplanned network outages in enterprise environments [3]. In addition, traditional networks don't give you a centralized view of network health—meaning admins usually only find out about a problem after something has already broken and service quality has dropped (a reactive approach).

So the central question this research tries to answer is: “How can we integrate open-source automation and monitoring tools to optimize the deployment and maintenance of network infrastructure in a way that cuts down our reliance on manual configuration while giving us full visibility into the network's state?” Without an accurate monitoring system and without automation, skilled staff end up wasting time on repetitive tasks, and the organization's operating expenses (OPEX) shoot up.

1.3. Project Objectives

The main goal of this project is to design and implement a modern architecture for managing network infrastructure, built on three pillars: standard design, configuration automation, and continuous monitoring. The specific objectives of this thesis are:

- **Designing scalable infrastructure:** implementing a layered network design model (Access, Distribution, Core) and rolling out basic network services like DHCP and DNS on Windows Server.
- **Implementing code-based automation (IaC):** using the Ansible framework to develop playbooks that configure network devices (switches and routers) automatically, repeatably, and without errors.
- **Removing the human-error bottleneck:** standardizing the configuration process through centralized management of YAML files.
- **Improving network observability:** deploying the Zabbix monitoring system to collect data in real time, monitor bandwidth, port status, and server resources, and set up proactive alerting.
- **Simulating an enterprise environment:** building an isolated testbed in GNS3 that closely mimics how real network equipment behaves.

1.4. Thesis Structure

To cover the objectives above thoroughly, this document is organized into six chapters. After this one (the introduction), Chapter Two reviews the literature and the foundational concepts, including network architecture, automation principles, an introduction to Ansible and Zabbix, and the idea of Infrastructure as Code. Chapter Three covers the first phase of the project—designing the network topology, configuring VLANs, and setting up the Windows Server services. Chapter Four (Phase Two) looks at deploying Ansible, developing playbooks, and automating the devices. Chapter Five (Phase Three) is about integrating the Zabbix monitoring platform and how the network elements are monitored. Finally, Chapter Six wraps up the results, discusses the technical challenges that came up along the way, and offers suggestions for future development.

Chapter Two: Literature Review and Foundational Concepts

2.1. Chapter Introduction

Before getting into the hands-on phase and walking through how the proposed architecture was built, it's worth taking the time to really understand the underlying concepts, protocols, and technologies used here. This chapter covers the theory behind modern computer networks, the principles of logical and physical design, and the shift from traditional methods to newer paradigms like Infrastructure as Code (IaC) and network automation. It also looks at the key tools and platforms used in the project—the GNS3 emulator, the Ansible automation framework, and the Zabbix monitoring platform—from a software-architecture angle and in terms of where they fit in enterprise environments.

2.2. Computer Network Architecture and Design

Designing a network that's stable, scalable, and secure means following standard, proven engineering models. In large enterprise networks, the flat network design approach has become obsolete because of problems like heavy broadcast traffic, lack of flexibility, and how hard it is to troubleshoot [4]. Instead, hierarchical models are used, which break the network into logical layers with clearly defined jobs.

2.2.1. The Hierarchical Network Model

Cisco, as one of the pioneers of the networking industry, introduced a three-layer design model that's now treated as a de facto standard in infrastructure architecture. This model has three distinct layers:

- **Access Layer:** This is where end-users, systems, servers, and peripheral devices connect to the network. Here, access switches handle basic security policies like Port Security, assign VLANs, and manage device power through PoE (Power over Ethernet). The main goal at this layer is to provide dedicated bandwidth and isolate user traffic at Layer 2 (the Data Link Layer) of the OSI model.
- **Distribution Layer:** This layer sits between the access and core layers and is responsible for aggregating traffic from the access switches. The most important networking operations happen here—inter-VLAN routing, traffic filtering based on Access Control Lists (ACLs), QoS policies, and route summarization. In effect, the distribution layer is the boundary that separates broadcast domains [5].
- **Core Layer:** The job of the core layer—the backbone of the network—is to move packets as fast as possible with the least delay. This layer avoids heavy processing tasks like ACLs or complex routing, so all the device's processing power can go toward fast switching. Reliability and redundancy are critical at this layer.

2.2.2. Virtual LANs (VLANs) and Related Protocols

The VLAN (Virtual Local Area Network) concept is one of the cornerstones of modern network design. VLANs let you logically split a single physical switch into several separate virtual switches. This logical separation has a bunch of benefits: smaller broadcast domains, better security by isolating traffic between different departments, and easier resource management. For network devices to carry traffic from multiple VLANs between each other, they need trunk links and the IEEE 802.1Q encapsulation standard.

2.3. GNS3 Network Simulation and Emulation Environments

In infrastructure engineering projects, rolling out a design directly onto physical equipment (a production environment) without testing it first is risky. That's why using simulation environments to validate network architecture has become an industry standard.

2.3.1. The Difference Between Simulation and Emulation

It's important to understand the difference between simulators (like Cisco Packet Tracer) and emulators (like GNS3). Simulators just mimic the software behavior of devices in a limited environment and don't run a real operating system. Emulators, on the other hand, use virtualization technologies like Dynamips, QEMU, and KVM to run real network operating systems (like Cisco IOS, MikroTik RouterOS, or hardware firewalls) on the host computer's hardware [6].

2.3.2. The Role of GNS3 in This Research

GNS3 (Graphical Network Simulator-3) is a powerful platform for implementing complex networking concepts. In this project, GNS3 wasn't just used to test simple connectivity—it served as a full testbed for integrating network devices (switches and routers) with x86-based virtual machines, like the Windows Server and the Linux server (running Ansible and Zabbix). This integration makes the implemented scenario behave just like a real-world scenario inside an actual organization.

Even though GNS3, as an advanced emulator, runs real network operating systems, the host CPU's hardware limitations still affect how the devices behave. This tension between emulation and physical production environments, and its impact on network protocols, is examined in detail in the technical evaluation in **Section 6.5 of Chapter Six**.

2.4. The Evolution of Network Management: Moving to Infrastructure as Code (IaC)

The traditional way of managing networks—manual configuration through the CLI—doesn't scale as networks grow. With this approach, engineers have to log into each device one at a time just to make a simple change, which is not only slow but also massively increases the chance of human error from mistyping commands or forgetting settings.

2.4.1. The Concept of Infrastructure as Code

The IT industry's answer to this challenge was Infrastructure as Code (IaC). IaC is a process where physical and virtual infrastructure is provisioned and managed through machine-readable code

instead of manual, interactive steps. In this paradigm, network configuration is treated like software code—meaning you can store it in version control systems (like Git), track the history of changes, and run CI/CD processes on it [7].

2.4.2. The Idempotency Principle

One of the most important features of IaC systems is idempotency. It guarantees that no matter how many times you run a configuration on a device, the end result is always the same. If the device's current state already matches the desired state defined in the code, the system makes no changes at all. This is vital in network automation because it stops re-running configuration scripts on live equipment from causing disruptions, and it keeps the service stable. This approach is the foundation for stepping into the world of NetDevOps.

2.5. The Ansible Automation Platform and Agentless Architecture

Among configuration management and automated deployment tools, Ansible (developed by Red Hat) has earned a special spot in enterprise environments thanks to how easy it is to learn and its unique architecture. Unlike tools like Puppet or Chef that need an agent installed on every target device, Ansible uses an agentless architecture [8].

2.5.1. The Communication Mechanism and the Push Model

Ansible's architecture is built on a push model—the central server (Control Node) pushes configurations out to the network devices (managed nodes). On Linux machines and network devices, this connection generally happens over the secure SSH protocol, and on Windows machines over WinRM. Features like SSH multiplexing let you connect to hundreds of devices at once with minimal processing overhead. In sensitive networks where installing third-party software on routers and switches isn't allowed for security reasons, this is a real competitive advantage.

2.5.2. The Logical Structure: Playbooks and Modules

Ansible's execution engine works on a declarative approach. In files called playbooks (written in the human-readable YAML language), the user describes the device's desired end state, and it's Ansible's job to compile and run the commands needed to reach that state, using purpose-built modules (like ios_config for Cisco devices or mikrotik_routeros for MikroTik routers) [9]. This structure lets network engineers turn even the most complex topologies into standard code—without needing deep programming knowledge—and keep them in version control systems.

2.6. Comprehensive Monitoring Systems and an Introduction to Zabbix

In the network lifecycle, once the design and deployment phases are done, the monitoring and maintenance phase becomes critically important. The point of monitoring is to move from a reactive approach to a proactive one—where the system warns admins about anomalies before a full outage even happens [21].

2.6.1. The Zabbix System Architecture

Zabbix is a unified, open-source, enterprise-class monitoring solution capable of tracking millions of metrics from networks, servers, virtual machines, and cloud services. The Zabbix architecture is made up of several components:

- **Zabbix Server:** the central processing core that collects data (polling/trapping), evaluates critical conditions (triggers), and sends alerts.
- **Zabbix Database:** a relational database (like MySQL or PostgreSQL) that holds all the settings, historical data, and events.
- **Zabbix Frontend:** the web-based interface used for data visualization, drawing graphs, and managing configurations.
- **Zabbix Proxy:** in distributed networks, proxies collect data from local devices and send it back to the central server in aggregate, which dramatically reduces the load on the main server.

2.6.2. Data Collection Mechanisms

In this architecture, network devices are mostly monitored through SNMP (Simple Network Management Protocol). Using the secure versions of this protocol (like SNMPv3) allows for encryption and authentication of monitoring traffic. For Windows and Linux servers, the Zabbix Agent is used, which pulls OS-level data (like processes, memory usage, and storage) with very high precision.

2.7. Centralized Infrastructure Services (Windows Server: DHCP & DNS)

No enterprise network can serve clients properly without basic addressing and name-resolution services. In this thesis, these critical services were set up on Microsoft Windows Server to match production environments as closely as possible.

2.7.1. Dynamic Host Configuration Protocol (DHCP)

DHCP is responsible for automatically assigning IP addresses, gateway addresses, and network parameters to clients. The address allocation process happens through four basic steps, abbreviated as DORA (Discover, Offer, Request, Acknowledge). Setting this service up centrally on the server prevents IP conflicts and makes it easier to manage the various subnets in the network's VLAN-based structure.

2.7.2. Domain Name System (DNS)

DNS acts as the network's phone book, translating human-readable domain names into the IP addresses the network uses. In environments where you plan to deploy automation tools (like Ansible), having a stable DNS server is critical, since automation scripts usually call devices by hostname or FQDN (Fully Qualified Domain Name) [8]. That said, a strategic engineering decision was made in this project's architecture: **separating the infrastructure control plane from the name-resolution service**.

To save processing resources, avoid circular dependencies, and make sure the Ansible engine could still manage devices even if the Windows (DNS) server went completely down, the infrastructure nodes in the inventory file are called directly by **static IPs**. The DNS service in this network is configured exclusively to handle communication for the department clients and access to web services, keeping the network consistent at the user layer.

Chapter Three: Phase One – Network Infrastructure Architecture, Design, and Deployment

3.1. Chapter Introduction

Any automation or monitoring system at the enterprise level depends absolutely on a stable, structured, and scalable underlay network. This chapter covers the first phase of the project in detail—designing the topology, allocating logical resources, configuring the access and distribution layers, and deploying the basic services. In this implementation, the goal was to avoid unnecessary complexity at the physical layer and focus on an optimal logical architecture, so the network would not only isolate departments precisely but also be ready to take on the Infrastructure as Code (IaC) scripts.

3.2. Network Topology Analysis and IP Addressing Scheme

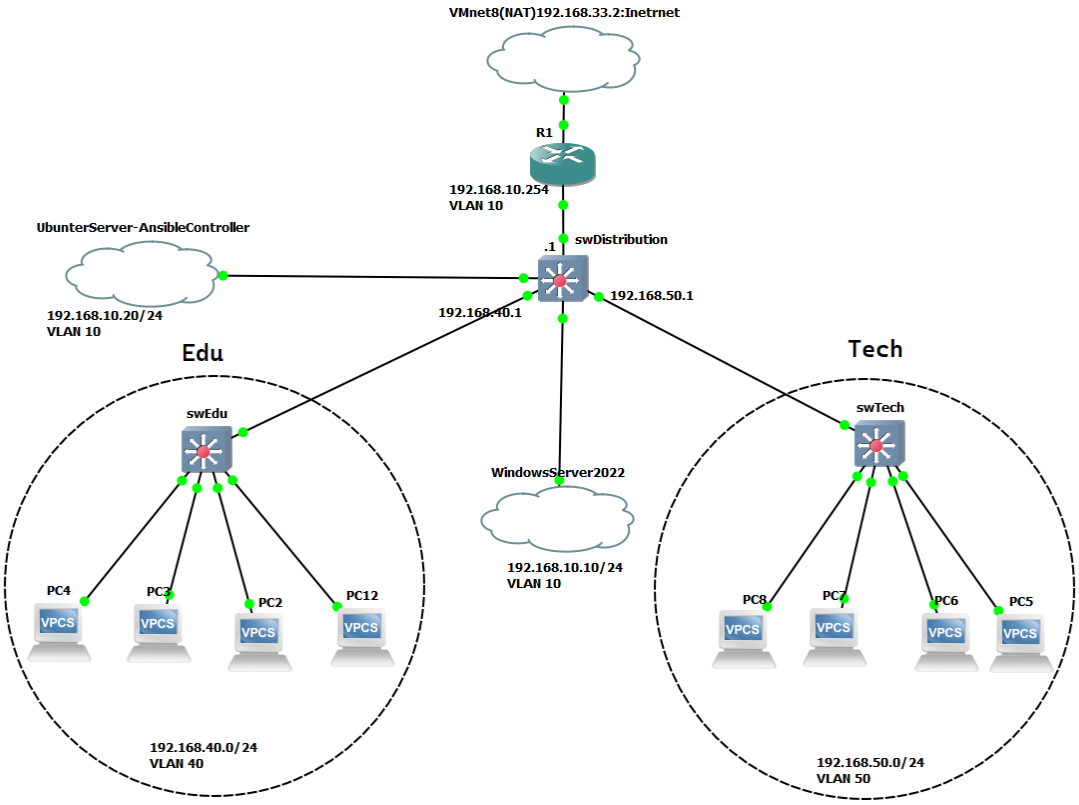

Figure 3-1: Logical and physical network topology in the GNS3 environment

As you can see in the network schematic (the topology diagram), this project's architecture is based on a two-layer (Collapsed Core) model. In this scenario, there are two main departments—Education (Edu) and Technical (Tech)—each with its own traffic and security requirements. The IP planning was done so that route summarization would be possible down the line and troubleshooting could happen quickly. The networks were assigned as follows:

- **Management & Infrastructure – 192.168.10.0/24:** hosts the critical servers, like Windows Server 2022 at 192.168.10.10 and the Ubuntu-based Ansible controller at 192.168.10.20.
- **Education Department – 192.168.40.0/24:** connected to the swEdu access switch.
- **Technical Department – 192.168.50.0/24:** connected to the swTech access switch.
- **Edge Connection:** using the VMnet8 network (192.168.33.0/24) in NAT mode for isolated communication with the global internet.

Table 3-1: IP address allocation matrix and the organization's logical VLAN structure

Device / Service	Interface / VLAN	IP Address	Subnet Mask	Gateway	Operational Role
Edge Router (R1)	To VMnet8	192.168.33.x	Dynamic / NAT	192.168.33.2	Outbound to internet
Edge Router (R1)	To swDist	192.168.10.254	255.255.255.0	—	Internal exit gateway
swDistribution	SVI - VLAN 10	192.168.10.1	255.255.255.0	192.168.10.254	Management gateway
swDistribution	SVI - VLAN 40	192.168.40.1	255.255.255.0	192.168.10.254	Education gateway
swDistribution	SVI - VLAN 50	192.168.50.1	255.255.255.0	192.168.10.254	Technical gateway
Ubuntu Server	VLAN 10	192.168.10.20	255.255.255.0	192.168.10.1	Ansible node / Docker host
Windows Server	VLAN 10	192.168.10.10	255.255.255.0	192.168.10.1	DNS and DHCP server
Education Clients	VLAN 40	192.168.40.0/24	255.255.255.0	192.168.40.1	PC2, PC3, PC4, PC12
Technical Clients	VLAN 50	192.168.50.0/24	255.255.255.0	192.168.50.1	PC5, PC6, PC7, PC8

3.3. Data Link Layer Strategy: Managing VLANs and Layer 2 Security

One of the most important architectural decisions in this project was keeping the access-layer switches (swEdu and swTech) limited to strict Layer 2 operation. With this approach, the access switches don't deal with Layer 3 (IP) packet processing at all—they only forward Ethernet frames [10]. This was done for a few reasons:

- Reducing CPU/RAM overhead on the user-edge devices.
- Concentrating security policies and ACLs on the distribution switch (swDistribution).
- Simplifying the network automation process in later phases.

3.3.1. Separating Broadcast Domains

To prevent traffic conflicts and improve security, the ports connected to workstations (PCs) were placed manually (static access) into their respective VLANs. Clients PC2 through PC12 in the

Education area were assigned to **VLAN 40 (Edu)**, and clients PC5 through PC8 in the Technical department to **VLAN 50 (Tech)**.

3.3.2. Securing the Native VLAN

In the 802.1Q protocol, frames belonging to the Native VLAN cross trunk links untagged. By default on Cisco devices, this is set to VLAN 1, which is a serious security risk for VLAN hopping attacks [11]. To avoid this vulnerability, the Native VLAN on all trunk links between the access and distribution switches was changed and moved to an unused blackhole VLAN, preventing any leakage of unmanaged data.

3.3.3. Implementing the Switch Virtual Interface (SVI)

Since the access-layer switches in this design don't have routing capabilities, they needed a management IP address for remote management over SSH (in preparation for deploying Ansible in the next phase). For this, **VLAN 10** was defined as the management network, and a virtual interface (SVI) was created on swEdu and swTech exclusively for VLAN 10, with management IPs assigned to them. This way, user traffic (the data plane) was fully separated from management traffic (the control/management plane).

3.4. Physical and Data Link Layer Challenges: Troubleshooting a Duplex Mismatch

While connecting the links between switches, one of the classic and important networking problems showed up: a duplex mismatch. This happens when auto-negotiation fails on both ends of a physical link—for example, one end is manually set to full-duplex while the other is in auto mode, and because it doesn't receive negotiation signals, it falls back to the network default of half-duplex [12].

Technical analysis and solution: In this situation, the port set to full-duplex sends frames without checking the line state (carrier sense). At the same time, the half-duplex port—which follows CSMA/CD (Carrier-Sense Multiple Access with Collision Detection)—sees this simultaneous transmission as a collision or a late collision and drops the frames. This caused a sharp drop in bandwidth, higher latency, and intermittent packet loss between the access and distribution switches. To fix the root cause and keep the trunk links stable for carrying sensitive traffic, auto-negotiation was disabled on the uplink ports, and speed and duplex were hardcoded to speed 1000 and duplex full on both ends. The result was completely stable Layer 2 connectivity.

3.5. Layer 3 Architecture and Centralizing the Exit Gateways

One of the fundamental challenges in designing multi-segment networks is deciding where to put the default gateway for clients. In this project, to optimize routing and lighten the load on the access layers, a “centralized gateway on the distribution switch” architecture was chosen.

3.5.1. Implementing Switch Virtual Interfaces (SVIs) on swDistribution

Unlike traditional models that hand routing off to the edge routers, here the Layer 3 switch (swDistribution) acts as the network's brain at Layer 3. For each VLAN defined in the previous phase (Edu, Tech, and Management), an SVI was created and assigned the IP corresponding to its subnet [13]. This approach came with several technical benefits:

- **Wire-speed switching:** inter-VLAN routing happens inside the switch's hardware (ASIC), which keeps latency to a minimum.
- **Reduced uplink traffic:** traffic between users on a single distribution switch no longer needs to leave the switch and go up to a router (the router-on-a-stick method).
- **Client stability:** the access-layer clients (swEdu and swTech) see the distribution switch as their gateway. So even if the link to the edge router (R1) goes down, internal communication (like access to local servers) keeps working.

3.6. Connecting to the Global Network and Managing the Edge Router (Edge Routing)

In this topology, router R1 acts as the gateway of last resort to the internet. It connects to the distribution switch through a Layer 3 link, and through another interface to the VMnet8 network (the NAT environment in the hypervisor).

3.6.1. Configuring Network Address Translation (NAT/PAT) on the Edge Router

To finish the first phase and get the network out of its isolated state, deploying address translation on the edge router (R1) was a must. Since this network's design uses private IPs, they can't be routed directly on the internet.

To solve this and let the internal nodes (especially the Ubuntu server, which needs to download software packages in later phases) reach the global network, NAT Overload (PAT – Port Address Translation) was configured on R1. In this setup, the interface connected to the distribution switch was set as ip nat inside and the interface facing the internet (VMnet8) as ip nat outside. Then, with an ACL defined, all outbound traffic from the internal subnets was translated to the external interface's IP. The CLI commands applied to R1 were:

```
R1(config)# access-list 1 permit 192.168.0.0 0.0.255.255
R1(config)# interface GigabitEthernet0/1
R1(config-if)# ip nat inside
R1(config)# interface GigabitEthernet0/0
R1(config-if)# ip nat outside
R1(config)# ip nat inside source list 1 interface GigabitEthernet0/0 overload
```

3.6.2. Static Routing Analysis and the ARP Challenge in Ethernet Environments

When configuring the default route on R1 for internet access, one critical networking point came up: “choosing between the exit interface and the next-hop address.”

For every node in the network (including the distribution switch and the clients) to be able to send traffic out, the default route has to be known throughout the whole network. In this scenario, a static route was written on swDistribution that sends unknown traffic toward R1's Layer 3 interface. This created a routing hierarchy where traffic flows from the lower layers up to the distribution layer and finally out to the internet through the edge router.

3.7. Deploying Processing Nodes and Infrastructure Servers in the Management VLAN

For centralized management and to set up the automation and monitoring phases, two strategic servers were placed in the Server Farm (in VLAN 10). This VLAN was chosen to isolate management traffic from general user traffic.

3.7.1. Ansible Controller Server (Ubuntu Server)

A virtual machine running the Ubuntu GNU/Linux distribution was set up as the Ansible control node. Linux was chosen for this role because automation tools are native to the Linux kernel and it handles SSH connections efficiently. This server was configured at 192.168.10.20, and the access it needed to reach all network devices (switches and routers) was guaranteed by defining return routes at Layer 3.

3.7.2. Microsoft Infrastructure Server (Windows Server 2022)

This server was deployed to provide directory, DNS, and DHCP services on the network.

- **DHCP service and the relay agent mechanism:** Since the server is in VLAN 10 but needs to hand out addresses to clients in VLAN 40 and VLAN 50, the IP Helper-Address feature was used on the distribution switch's SVIs. This command turns the clients' broadcast requests into unicast and forwards them to the Windows Server [14].

3.8. Stability and Integrity Analysis (Post-Implementation Analysis)

After the configurations above were complete, the network went into stress and stability testing. Using traffic analysis tools (like Wireshark in GNS3), the correct flow of packets between the different VLANs and through the NAT layer was verified. The results showed that placing the gateways at the distribution layer qualitatively improved round-trip time and left the network fully ready to take on complex NetDevOps scenarios in the next phases.

Chapter Four: Phase Two – Network Automation Architecture and Code-Based Configuration Deployment

4.1. Chapter Introduction

As IT infrastructure keeps growing and organizations urgently need agility in delivering services, the traditional CLI-based way of managing networks has become an operational bottleneck. In the previous chapter, the network's communication foundation was laid following hierarchical design principles. This chapter is the turning point of the research—where the software-defined networking (SDN) paradigm and DevOps engineering approaches meet to create the concept of NetDevOps. In this phase, the Ansible automation platform is set up as the network's control plane, and configuration management of the access- and distribution-layer devices moves from static, manual work to a fully automated, repeatable, text-file-based structure (Infrastructure as Code).

4.2. Moving from Interactive Management to a Declarative Approach

In traditional network management, the engineer uses an imperative approach—entering commands step by step to reach a goal. But Ansible is built on a declarative approach. Here, the user only describes the device's "desired end state" (for example: a specific VLAN with a specific name should exist), and Ansible's engine figures out the difference between the device's current state and the desired state, then issues only the commands needed to close that gap [15]. This feature—called idempotency—stops destructive or repetitive changes from happening when scripts are re-run, and keeps the network stable.

4.3. Preparing and Architecting the Control Node

In Ansible's client-server architecture, the system the scripts run from is called the control node. In this project, a virtual machine running Ubuntu Server (in the management network at 192.168.10.20) was assigned this role.

4.3.1. The Critical Role of the Python Interpreter

Unlike Windows-based operating systems, Linux distributions support automation natively. At its core, Ansible is built with Python. Running the network modules and processing YAML files depends entirely on the Python runtime environment on the control node. In this implementation, before installing the Ansible core, the base Python libraries and the Python package manager (PIP) were updated to ensure full compatibility with the Cisco network modules [16].

4.3.2. The Secure Transport Mechanism

Since Ansible uses an agentless architecture, there's no need to install any intermediary software on the Cisco routers and switches (managed nodes). Communication between the Ubuntu server and the network devices happens exclusively over the secure application-layer protocol SSH. At the code level, Ansible uses low-level Python libraries like Paramiko, to manage SSH connections with minimal processing overhead [8].

4.4. The Connection Layer Architecture for Network Devices (Network Connection Plugins)

One of the fundamental differences between automating network devices and automating Linux servers comes down to the nature of network operating systems like Cisco IOS. On Linux servers, Ansible sends Python code straight to the target server and runs it there (the ssh plugin). But traditional network devices don't have a Python interpreter.

4.4.1. The Network_CLI Plugin

To get around this limitation on network devices, Ansible uses a special connection plugin called network_cli. In this research, the ansible_connection: network_cli parameter was set in the configuration. With this approach, instead of sending Python code to the switches, the control node opens an SSH connection at the terminal (CLI) level, parses the device's text output (prompt), and sends the matching Cisco commands natively. The ansible_network_os: ios parameter explicitly tells the engine to interact with Cisco's syntax and behavior, which prevents command-syntax errors.

4.5. Designing the Data Model: The Inventory System

The first operational step in running playbooks is defining the devices that will be managed. This information is stored in a text-based database called the inventory.

4.5.1. Designing the Inventory Data Model and Explaining the Interactive Behavior Variables

To introduce the network devices to the Ansible control node, a centralized inventory file in the standard INI format, named inventory.ini, was developed. In this data structure, devices are split into separate groups based on their operational role and where they sit in the topology: [routers], [core_switches], and [access_switches]. To manage things efficiently and avoid repeating configuration code, group inheritance was used by defining a parent group [network_devices:children]. This way, all the subgroups inherit the management properties from the parent group. Code snippet 4-1 shows the final, working structure of this inventory file.

Code 4-1: The inventory file structure (inventory.ini) and the logical grouping of devices by IP

```
[routers]
R1 ansible_host=192.168.10.254

[core_switches]
swDistribution ansible_host=192.168.10.1

[access_switches]
swEdu ansible_host=192.168.10.11
swTech ansible_host=192.168.10.12

[network_devices:children]
routers
core_switches
access_switches

[network_devices:vars]
ansible_user=admin
ansible_password=123
# WARNING: Credentials in plaintext for lab only.
# In production, encrypt using Ansible Vault:
# ansible-vault encrypt_string 'your_password'

ansible_network_os=ios
ansible_connection=network_cli
ansible_become=yes
ansible_become_method=enable
ansible_become_password=
```

4.5.2. Breaking Down the Behavioral Parameters and Secure Sessions in the Automation Layer

The engineering value of this inventory file lives in the global variables block, [network_devices:vars]. These parameters set how the SSH sessions behave, authenticate, and are encapsulated:

- **ansible_host:** this key variable maps the device's logical alias (like R1 or swEdu) directly to its static management IP. Using it means the Ansible engine doesn't need DNS or OS-level hostname configuration to open SSH sessions—it communicates over the fixed, secure IPs defined here.
- **ansible_user and ansible_password:** these define the devices' local AAA authentication info centrally, so Ansible can use them when opening the SSH tunnel.
- **ansible_network_os=ios:** this explicitly tells the Ansible engine that the target devices run Cisco IOS, which loads Cisco's native command plugins when the YAML code is compiled.
- **ansible_connection=network_cli:** unlike Linux servers that use standard SSH connections, network devices need the network_cli plugin. This plugin creates a persistent SSH session that stays open through all the tasks, which dramatically cuts the overhead of repeatedly opening and closing secure tunnels.

- **Privilege escalation (ansible_become):** since applying settings on Cisco devices requires privilege exec mode, ansible_become=yes was enabled. With ansible_become_method=enable, Ansible knows it has to issue Cisco's native enable command to escalate. Leaving ansible_become_password empty means no separate enable-mode password was set in the lab environment, which speeds up the automated runs.

4.5.3. Logical Grouping and Variable Hierarchy

For efficient management, the network devices in the inventory were split into logical groups. For example, the access-layer switches (swEdu and swTech) went into a group called [access_switches], and the distribution switch into [core_switches]. This logical grouping lets the admin assign common variables (like username, password, and OS type) to an entire group through the group_vars folder, instead of defining them for each device one by one. This significantly cuts down repetitive code (DRY: Don't Repeat Yourself) and greatly improves the project's readability and maintainability.

4.6. Breaking Down the Code Architecture and Implementing the Operational Scenarios

With the control node and the inventory file done, the infrastructure development and coding phase began. The project's files and folders were laid out in a modular directory structure. All the playbooks were organized into a separate directory called playbooks to respect the separation-of-concerns principle from software engineering [8]. The implementation strategy was to develop independent, single-responsibility playbooks. Based on that, five separate scenarios were developed as YAML files:

1. Dynamic backup (Automated Network Backup)
2. Baseline standardization (Standardize Configuration)
3. VLAN provisioning (Provision VLANs)
4. Security hardening (Security Hardening)
5. Monitoring preparation (Monitoring Preparation)

In all these scripts, the gather_facts: false parameter was set on purpose. Since gathering system facts on network devices is slow and puts extra load on the routers and switches, disabling it in scenarios that don't need system variables (like the device serial number) noticeably optimizes the code's execution time [1].

4.7. Scenario One: A Dynamic, Automated Backup System (Dynamic Backup Automation)

One of the fundamental challenges in managing enterprise networks is having the latest copy of the device configurations (running-config) before making any change. The 1_backup.yml file was developed to handle this need and create an automated archiving system.

4.7.1. Workflow Analysis

Instead of blindly firing off commands, this playbook uses a multi-step logic that leans on the Jinja2 templating engine:

- **Timestamping:** first, using the command module to call the Linux date command, the system's exact date and time are pulled and registered in a variable called date_var.
- **Local delegation:** the key engineering point here is the use of delegate_to: localhost. This tells Ansible that creating the folder (with the file module) shouldn't happen on the Cisco switches, but on the Ubuntu server (the control node) itself. This technique is an architectural necessity for hybrid processes that need interaction between the host OS and the network devices [8].
- **Extraction and saving:** finally, the ios_command module runs show run on the network devices and dumps the output into temporary memory. Then the copy module saves it to the pre-created path (using Jinja2 mapping like {{ date_var.stdout }}) under each device's own name ({{ inventory_hostname }}.txt). This cycle completely removes any dependence on commercial backup software like SolarWinds NCM.

Code Snippet 4-2: The backup playbook

```
---
- name: Scenario 1 - Automated Network Backup
  hosts: network_devices
  gather_facts: false
  tasks:
    - name: Get Date
      command: date +%Y-%m-%d_%H:%M
      register: date_var
      delegate_to: localhost
    - name: Create Directory
      file:
        path: ./backups/{{ date_var.stdout }}
        state: directory
      delegate_to: localhost
    - name: Get Config
      ios_command:
        commands: show run
      register: config_output
    - name: Save to File
      copy:
        content: "{{ config_output.stdout[0] }}"
        dest: "./backups/{{ date_var.stdout }}/{{ inventory_hostname }}.txt"
   delegate_to: localhost
```

4.8. Scenario Two: Implementing a Baseline and Standardization (Baseline Standardization)

After securing the data with a backup system, the 2_Standardize.yml file runs to bring the base settings across the entire access and distribution layers in line. The goal of this scenario is to make sure uniform policies are applied to all the nodes.

4.8.1. Declarative State Management

This script uses the powerful ios_config module. Instead of typing commands at the CLI, this module treats the device's configuration as a "state."

- **Encryption and DNS:** commands like service password-encryption (to keep passwords from showing up in clear text) and DNS server settings (ip name-server 192.168.10.10) are injected into the devices as lines. Using the local DNS server set up on Windows Server in Phase One lets the switches resolve domain names themselves.
- **Banner management (Banner MOTD):** using the dedicated ios_banner module, an "Authorized Access Only" warning was configured on all the devices. Using the | character in YAML allows multi-line strings while preserving the visual structure, which is needed for designing standard enterprise banners.

4.8.2. Idempotency in Action

The crown jewel of Ansible's architecture sits in the last line of this scenario, where the save_when: modified parameter is used in the ios_config module. In traditional networks, after every change the admin always issues write memory, which wears down the NVRAM over time and causes millisecond-level pauses in the switch's processor [15]. But this parameter tells Ansible: "First compare the device's running-config with the startup-config. If, and only if, a change was made in this run, save the changes to NVRAM." This is the real expression of idempotency in NetDevOps, and it keeps the network stable across repeated runs.

Code Snippet 4-3: The security standardization script

```
---
- name: Scenario 2 - Apply Standard Configuration
  hosts: network_devices
  gather_facts: false
  tasks:
    - name: Set login Warning Banner
      ios_banner:
        banner: motd
        text: |
          ##################################################
          #         WARNING: AUTHORIZED ACCESS ONLY        #
          #   All activity on this device is monitored.    #
          #       Disconnect IMMEDIATELY if unsure!        #
          ##################################################
        state: present
    - name: Encrypt Clear-text Passwords
      ios_config:
        lines:
          - service password-encryption
    - name: Configure DNS Server
      ios_config:
        lines:
          - ip name-server 192.168.10.10
          - ip domain-lookup
    - name: Save Configuration to NVRAM
      ios_config:
        save_when: modified
```

4.9. Workflow Integration and the Execution Logic of the Scenario Hierarchy

Before getting into the code architecture of the later scenarios, it's worth explaining the logic and strategy behind running these playbooks. In NetDevOps, running scripts isn't a random process—it follows a carefully engineered pipeline. In this project, the scenarios were designed so that each phase's output is the prerequisite for the next. First, running Scenario One (backup) locks in the device's current state as a safe restore point. Then Scenario Two standardizes the base platform and the initial access. After that foundation is set, Scenario Three (VLAN configuration) shapes the Layer 2 logical architecture. Finally, on top of that, Scenario Four hardens the network's security, and Scenario Five gets the infrastructure ready to connect to the centralized monitoring system (Zabbix) in the upcoming phases. This logical continuity significantly reduces the risk of configuration drift across all Ansible-managed parameters and simulates a CI/CD approach in the network infrastructure [9].

4.10. Scenario Three: Provisioning Logical Infrastructure and Data-Driven Management of Virtual Networks

The 3_vlan.yml file is responsible for deploying the network's Layer 2 architecture. But the engineering value of this script isn't just in creating a few simple VLANs—it's in the paradigm shift from static configuration to data-driven configuration.

4.10.1. Removing VTP and Moving the Source of Truth

In the first task of this playbook, the VTP (VLAN Trunking Protocol) mode on all switches was changed to transparent. In traditional networks, VTP is used to automatically distribute VLAN information between switches, but the protocol is prone to a dangerous vulnerability known as the “VTP Bomb”—where a new switch with a higher revision number can wipe out the organization's entire VLAN database [14]. In NetDevOps thinking, dynamic protocols like this are considered obsolete. By disabling VTP, we explicitly declared that the network's single source of truth is no longer the devices themselves, but our Ansible code and repository, which determine the network's final state.

4.10.2. Loop Architecture and Separating Data from Logic

In this script, the list of VLANs (Management, Edu, Tech, Native) isn't defined inside the command body, but as a dictionary data structure in the vars section (the my_vlans variable). Then, using the loop loop: "{{ my_vlans }}" and the ios_config module, the information is read dynamically and injected into the switch.

Engineering benefit: with this architecture, if the organization ever needs to add dozens of new VLANs, the admin doesn't have to touch or tweak the execution logic at all—they just add the new data to the variable list. This decouples execution logic from data, enabling the playbook to scale horizontally across an arbitrarily large device inventory without any modification to the task code.

Code Snippet 4-4: Ansible playbook for dynamically provisioning VLANs (data-driven architecture)

```
---
- name: Scenario 3 - Provision VLANs
  hosts: core_switches, access_switches
  gather_facts: false
  vars:
    my_vlans:
      - { id: 10, name: mgmt }
      - { id: 40, name: edu }
      - { id: 50, name: tech }
      - { id: 99, name: native }
      - { id: 71, name: Test }
  tasks:
    - name: Set VTP Mode to Transparent
      ios_config:
        lines:
          - vtp mode transparent
    - name: Create and Name VLANs with ios_config
      ios_config:
        parents: vlan {{ item.id }}
        lines:
          - name {{ item.name }}
      loop: "{{ my_vlans }}"
    - name: Save Config
      ios_config:
        save_when: modified
```

4.11. Scenario Four: Security Hardening and Reducing the Attack Surface

The 4_security.yml file is one of the most critical parts of the automation, developed to bring the devices in line with security standards (like the CIS Benchmarks). In large networks, configuring security settings by hand on each port and switch carries a high risk of human error and can create hidden vulnerabilities. Ansible guarantees that security policies are applied uniformly across the whole network.

4.11.1. Stopping Vulnerable Services and Controlling Discovery Protocols

Cisco devices' default services (like the HTTP server and BOOTP) are considered threats in modern networks because they don't support strong encryption. In this playbook, these services were disabled using the lines structure. Also, the Cisco Discovery Protocol (CDP)—which can leak sensitive topology info, OS versions, and IP addresses to attackers plugged into access-layer ports (reconnaissance attacks)—was globally turned off with the no cdp run command [17].

4.11.2. Securing Remote Access Lines (VTY & Console Lines)

One of the most important steps here is fully blocking the insecure Telnet protocol and forcing the devices to use SSH version 2 (SSHv2). This neutralizes the risk of sniffing attacks and credential theft. A notable point in developing this code is the deliberate awareness of the runtime environment—as the comments in the code correctly point out, the auto-disconnect time (exec-timeout) for the console port was set to 0 0 (unlimited) because of the project's lab nature, to avoid frequent disconnects during development. But thanks to careful documentation, the script is fully ready to be switched to industry-standard values (like 5 0) for production deployment.

Code Snippet 4-5: The security hardening script

```
---
- name: Scenario 4 - Security Hardening
  hosts: access_switches, core_switches
  gather_facts: false
  tasks:
    - name: Disable Unused Services (HTTP, BOOTP, CDP)
      ios_config:
        lines:
          - no ip http server
          # In real world also add: no ip http secure-server
          - no ip bootp server
          - no cdp run
          - service password-encryption
    - name: Enforce SSH Version 2 and Timeouts
      ios_config:
        lines:
          - ip ssh version 2
          - ip ssh time-out 60
          - ip ssh authentication-retries 3
    - name: Secure VTY Lines (kill Telnet)
      ios_config:
        parents: line vty 0 4
        lines:
          - transport input ssh
          - exec-timeout 5 0
          - logging synchronous
    - name: Configure Console Line for Lab (Never Timeout)
      ios_config:
        parents: line con 0
        lines:
          - logging synchronous
          - exec-timeout 0 0
          # Should be 5 0 in production, 0 0 in lab demo like here
    - name: Save Config
      ios_config:
        save_when: modified
```

4.12. Scenario Five: Instrumentation and Preparing the Monitoring Foundation

The 5_monitoring_prep.yml file is the missing link and the bridge between Phase Two (automation) and Phase Three (monitoring) of this project. A powerful monitoring system like Zabbix is completely useless without standard access to the network nodes.

This automation scenario (enabling SNMP on the switches) is really the cornerstone of Phase Three. The deep connection between this scenario's code and setting up the Zabbix observability platform is analyzed in **Section 5.6 of Chapter Five**, to show how automation is a prerequisite for monitoring.

4.12.1. Unified Configuration of SNMP

In this scenario, SNMP version 2c was enabled on all the access- and distribution-layer devices. The critical variables—including the community string named snmp_pass—were configured with limited, read-only (RO) access, so that even if the community string were exposed, an attacker couldn't change the infrastructure settings [24].

The benefit of doing this with Ansible: SNMP settings are the most error-prone part for human mistakes (like a typo in the monitoring server's address or the community string), which create “dark spots” in the organization's monitoring dashboards. By using Ansible and setting zabbixServer_ip: 192.168.10.20 as a global variable, we made sure all the switches connect to the Ubuntu server with exactly the same structure—without the slightest deviation—and are ready to send traffic data, port status, and CPU usage to the Zabbix server in the next phase.

Code Snippet 4-6: SNMP configuration script for integration with the monitoring system

```
---
- name: Phase 3 - Step 1 Enable SNMP on Devices
  hosts: network_devices
  gather_facts: false
  vars:
    snmp_pass: "public"
    # WARNING: This value is for lab/demo purposes only.
    # In production, use Ansible Vault to encrypt sensitive credentials.
    # Reference: https://docs.ansible.com/ansible/latest/vault_guide/index.html
    zabbixServer_ip: "192.168.10.20"
  tasks:
    - name: Configure SNMP Server
      ios_config:
        lines:
          - snmp-server community {{ snmp_pass }} RO
          - snmp-server host {{ zabbixServer_ip }} version 2c {{ snmp_pass }}
    - name: Save Config
      ios_config:
        save_when: modified
```

4.13. Analyzing the Execution Output and Validating the Desired State

Developing automation scripts is only half of NetDevOps; the other critical half is running them correctly and analyzing the system feedback. Once the five playbooks were done, the configuration deployment started using the ansible-playbook command from the Ubuntu server's command line.

4.13.1. Play Recap Analysis

When running playbooks against the network, Ansible's engine gives a summary report called the Play Recap after processing each node. On the first run of these scenarios, all the switches showed up with a changed status. This meant the devices' current state didn't match the desired state defined in the code, so Ansible went ahead and injected the commands into the device's memory [8].

The most important technical takeaway of this phase showed up on the second run. When the scenarios were run again (with no changes to the code), the engine—after checking the devices' state—reported them as ok (green in the terminal) with changed=0. This output is concrete,

practical proof that idempotency was successfully implemented in this project's network infrastructure—meaning the system reached absolute stability, and running the code repeatedly won't add any extra processing load or cause destructive changes.

4.13.2. Transport-Layer Error Handling

During execution, the default SSH timeout parameters needed tuning, given the processing limits of the GNS3 emulator. If Ansible doesn't get a response within the standard window, it stops the process with a timeout error. To get past this environmental challenge, the network environment variables in the Ansible core config file (ansible.cfg) were tuned, and the [persistent_connection] parameter was configured with a longer response time so the concurrent connections to the switches wouldn't drop under heavy load.

Chapter Five: Phase Three – Centralized Monitoring Architecture, Performance Management, and Building Observability

5.1. Chapter Introduction

Deploying a stable network and automating it with Infrastructure as Code tools is incomplete without a dynamic monitoring and observability system. The reality is, admins need a tool that checks the hardware and software health of devices in real time and raises the right alerts before critical failures cause downtime. In the previous phases, the network was designed and the security processes and baselines were deployed with Ansible. This chapter covers Phase Three of the project—setting up the centralized Zabbix monitoring platform on Docker containers and tuning it to collect and analyze critical network data.

5.2. Comparative Evaluation of Infrastructure Monitoring Platforms

Choosing a monitoring platform for a mixed enterprise network (with both Cisco devices and Linux/Windows servers) calls for a careful look at the technical criteria. To justify the choice of Zabbix from an engineering standpoint, four well-known industry solutions were analyzed:

- **Prometheus:** built on a time-series database (TSDB), it's excellent for cloud-native and containerized environments. That said, Prometheus uses a pull model over HTTP, while traditional network devices (like Cisco switches) don't have native Prometheus exporters. Using it would mean setting up complicated middleware (like the SNMP Exporter), which adds management overhead [19].
- **Nagios:** one of the oldest open-source tools, it does monitoring based on periodic scripts (plugins). Its biggest weakness is its traditional file-based architecture. Changing settings means manually editing heavy text files and restarting the service, which clashes with the agility NetDevOps demands.
- **PRTG Network Monitor:** a very powerful proprietary, commercial tool for network monitoring on Windows. While PRTG has an attractive UI, it was dropped from consideration in this project because it's closed-source, the free version is severely limited (capped at 100 sensors), and it can't integrate deeply with Linux automation pipelines.
- **Zabbix:** an enterprise-grade, fully open-source platform that was chosen as the final monitoring solution for this project for the following reasons:
- **Native SNMP support:** it can interact directly with the Cisco IOS core without any middleware.
- **Modular architecture:** complete separation of the processing layer (Zabbix Server), the presentation layer (Zabbix web), and the storage layer (database).
- **Dynamism and automation capability:** very powerful JSON-RPC-based APIs that let Ansible automate registering nodes in the monitoring system too.

5.3. The Containerization Paradigm: Why We Migrated to Docker

After choosing Zabbix, how to deploy it was the next architectural challenge. Installing software the traditional way, natively on the OS (bare-metal or traditional VMs), leads to infrastructure rigidity and dependency conflicts ("dependency hell") [22]. To solve this, Docker containerization was chosen as the main deployment platform.

5.3.1. What Is Docker, and How Does It Differ from Traditional Virtualization?

In traditional virtualization (like the hypervisor structure in VMware), each VM needs a full guest OS along with its own dedicated kernel, which wastes substantial RAM and CPU resources. Docker, on the other hand, is an open-source containerization technology that lets applications run inside fully isolated environments called containers. Unlike VMs, containers share the host OS kernel and use Linux kernel features like Namespaces (for isolating spaces) and Control Groups (cgroups, for limiting hardware resource usage) [22]. This makes containers extremely lightweight and reduces startup time from several minutes (for full VMs) to a few seconds.

5.3.2. The Benefits of Deploying Zabbix on Docker in This Project

Using Docker to set up Zabbix in this project brought the following strategic benefits:

- **Environment portability:** Zabbix needs a web server (Apache/Nginx), a PHP interpreter with with extensive PHP dependencies and ancillary libraries, and a database (MySQL/PostgreSQL) to run. With containerization, this whole software stack is deployed as pre-tested containers and behaves exactly the same in the lab and in production.
- **Easy lifecycle management:** upgrading the monitoring system down the line is just a matter of changing the image tag—done in seconds without harming the existing data.

5.4. Local Deployment Challenges: Getting Around International Sanctions and Migrating to Domestic Mirrors

While deploying Docker on the Ubuntu server (192.168.10.20), the project ran into one of the biggest and most exhausting operational challenges in domestic network infrastructure: the restrictions caused by international sanctions and reciprocal filtering.

5.4.1. Technical Analysis of the Docker Hub Blockage

The official Docker Hub server (registry.docker.io)—the main source for downloading standard images—blocks requests coming from Iranian IP blocks based on sanctions policies, hitting them with the classic HTTP 403 Forbidden error or TLS handshake timeouts. This completely halted the download of the critical Zabbix images.

Code Snippet 5-1: A sample transport-layer error when calling Docker Hub

```
Error response from daemon: Get "https://registry-1.docker.io/v2/":
net/http: TLS handshake timeout
```

5.4.2. The Engineering Solution: Configuring Docker Registry Mirrors

To get past this structural challenge and keep the deployment going without relying on unstable traffic-encryption tools (VPNs, which themselves caused a sharp drop in bandwidth and instability in the hypervisor environment), the Docker image-pull infrastructure was re-engineered. Docker on the Ubuntu server was set to use domestic mirror and intermediary servers (Iranian local mirrors) instead of pointing to the original servers. This was done by editing and applying native settings in the Docker daemon config file (/etc/docker/daemon.json) [23]. By adding the addresses of stable domestic mirrors—like ArvanCloud or Hamravesh—to the registry-mirrors array, the container download requests were routed first to these domestic intermediary servers (which act as a centralized cache), so the Zabbix images downloaded and deployed at full physical line speed. This is a clear example of managing an infrastructure crisis under technological sanctions.

5.5. Data Infrastructure Architecture and Ensuring Data Persistence

One of the main pillars in deploying enterprise monitoring platforms is designing the data storage layer. Zabbix continuously collects a huge volume of time-series data—including traffic metrics, hardware status, and system logs.

5.5.1. Choosing and Examining the Database in the Container Environment

In this project, a relational database (RDBMS) container was deployed as Zabbix's main storage engine. In Docker's architecture, if a database container is set up without special measures, the data is stored temporarily in the container's writable layer—meaning that when the container stops or is deleted, all the historical monitoring data (History & Trends) is wiped out completely [22].

5.5.2. Implementing Docker Volumes for Data Persistence

To overcome the data ephemerality problem, the Docker Volumes mechanism was used. In the docker-compose.yml config file, a physical directory from the host OS (Ubuntu Server) was mounted to the database container's internal directory (/var/lib/postgresql/data). This decoupling of compute and storage guarantees that even if the database container fails or gets upgraded, the data structure and network tables stay completely intact. Environment variables including credentials were defined in the Compose file. For lab purposes, these are stored in plaintext. In production environments, Docker Secrets or an external vault should be used to protect sensitive values.

5.6. Analyzing the Structural Link Between the Automation and Monitoring Phases

The relationship between Phase Two (automation) and Phase Three (monitoring) is one of cause and effect, and it's synergistic. To collect data from the access- and distribution-layer devices, Zabbix depends entirely on SNMP.

5.6.1. The Role of Ansible's Fifth Scenario in Enabling Monitoring

Without running 5_monitoring_prep.yml in the automation phase, centralized network monitoring would have been impossible. In that phase, Ansible played the role of an infrastructure catalyst. Running that playbook automatically and simultaneously applied the following on all the switches:

- Defining a read-only community string (snmp-server community snmp_pass RO).
- Configuring the SNMP notification destination to point exclusively to the Zabbix server's IP (192.168.10.20) using the snmp-server host directive.

5.6.2. The Engineering Benefit of Eliminating Configuration Drift

If this had been done traditionally and by hand, one small typo in the community string or the monitoring server's IP on any single switch would have created a “network blind spot”. Ansible's idempotent execution guaranteed a uniform communication baseline across all nodes, eliminating the per-device inconsistencies inherent in manual CLI configuration. As a result, as soon as the containers came up, Zabbix could immediately start polling—without needing any local reconfiguration on the switches [8].

5.7. Engineering the Centralized Dashboard and Analyzing Live Metrics

The Zabbix dashboard acts as the command-and-control center for this project's entire infrastructure. This web interface turns the raw data coming in from the network protocols into understandable visual layers.

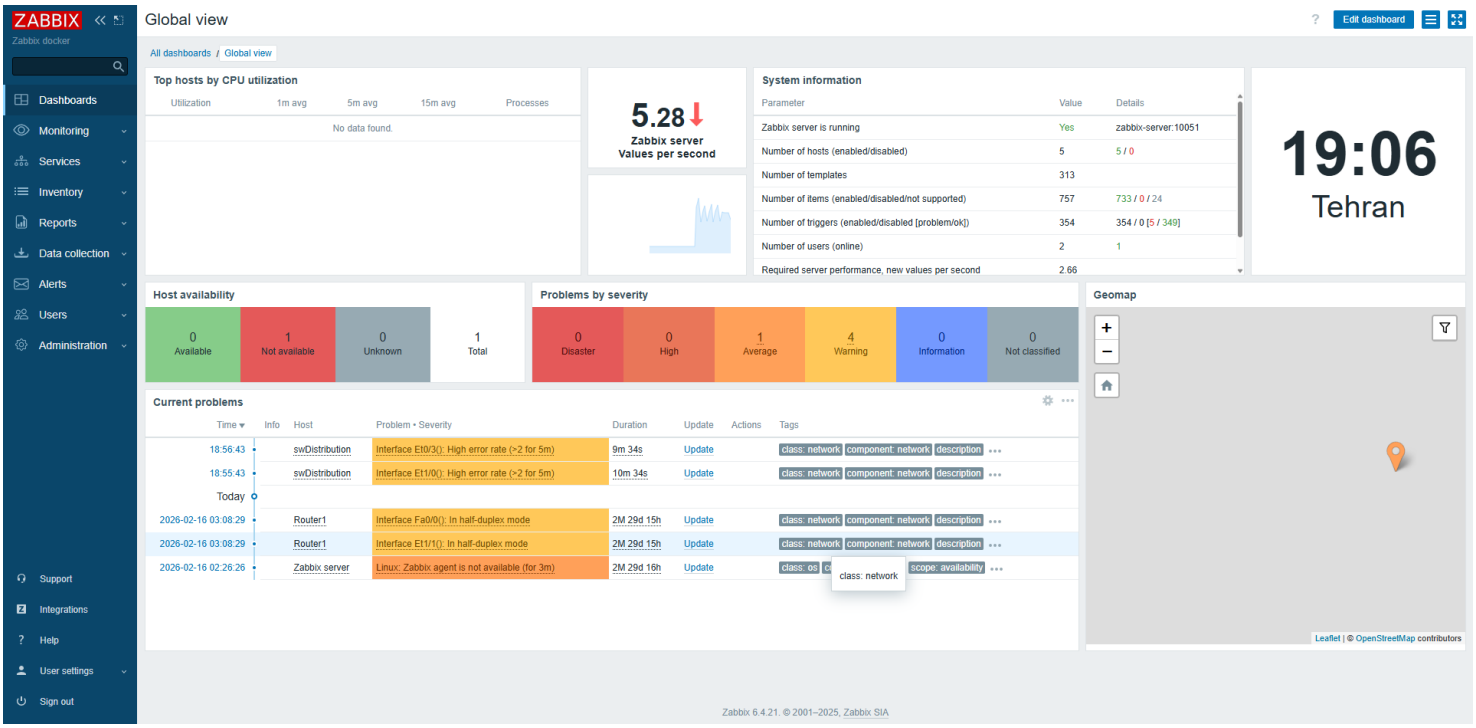

Figure 5-1: The centralized Zabbix monitoring dashboard and the stability status of the network nodes

The dashboard was configured to show the admin the following at a glance:

- A device health status table (System Status) based on standard color codes (green for normal, yellow for warning, red for critical).
- The rate of events (System Top Triggers) to spot the nodes carrying the most traffic load or errors.
- A live availability chart for the devices based on the ICMP and SNMP protocols.

5.8. Examining the Telemetry Transport Layer: Why SNMP?

To collect critical metrics from network hardware, you need a standard, lightweight, universal protocol at the application layer. SNMP (Simple Network Management Protocol) was chosen as the industry's gold standard for this [24].

5.8.1. SNMP's Information Architecture: The MIB and OID Concepts

Understanding how Zabbix works in this phase means explaining the internal architecture of the network devices' database. Every Cisco device stores its information in a hierarchical tree structure called the **MIB (Management Information Base)**. Each point or leaf in this tree has a unique numeric address called an **OID (Object Identifier)** [25]. For example, the CPU usage over a given window, or the traffic status of a specific port, each has its own dedicated OID (like 1.3.6.1.2.1.2.2.1.10).

The Zabbix server (the SNMP Manager) periodically sends out its requests as SNMP GetRequest packets over the unreliable but fast UDP protocol (port 161) to the switches (the SNMP Agents), and the switch pulls the data from its MIB and sends back a response. This polling process keeps the monitoring continuous.

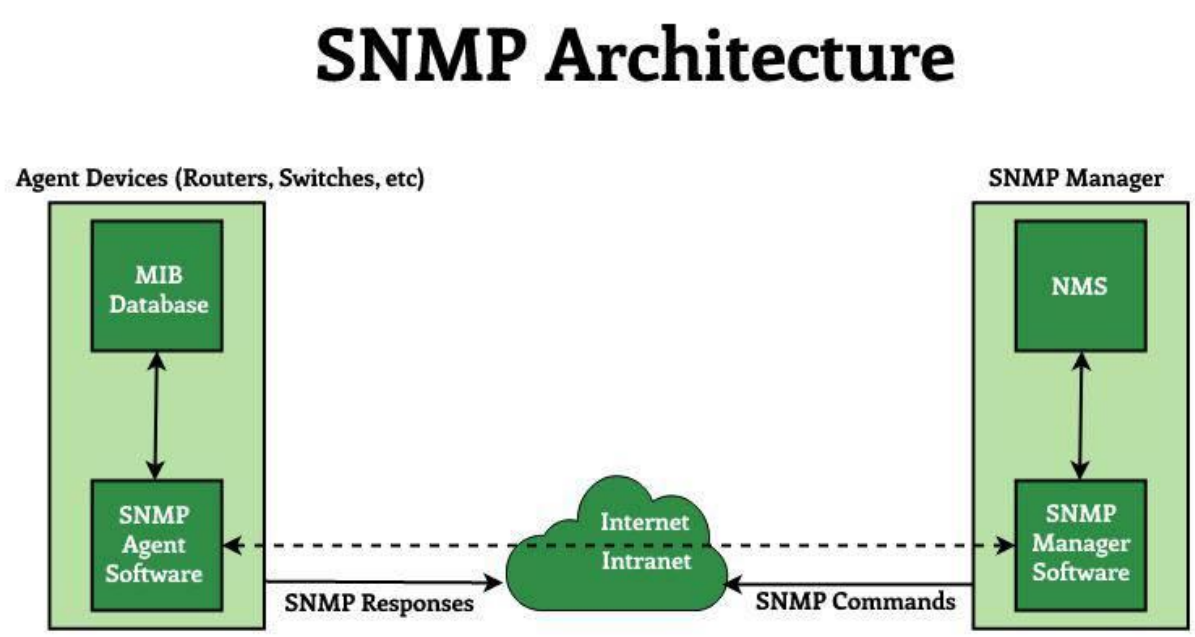

Figure 5-2: The high-level communication architecture of SNMP and the data flow between the Manager and the Agent [26]

5.9. The Operational Monitoring Subsystem and Node Availability Analysis

After Ansible injected the baselines, the interactive foundation was set at the transport layer. At this stage, the network nodes were registered in Zabbix's monitoring menu under Monitoring → Hosts. This section, as the live status layer, is responsible for continuously monitoring and instantly displaying the devices' availability.

5.9.1. Analyzing the Availability Indicator Mechanism

At this layer, Zabbix shows each device's connectivity status through color codes in the availability matrix (the SNMP section).

- **Green indicator (operational state):** shows that the monitoring server has full access to the switch's or router's MIB database. Here, SNMP Get-Request packets are sent at the defined intervals and the matching responses come back without packet loss.
- **Red indicator (failure state):** this means there's a break in the communication path or at the service layer. The indicator turning red shows that the “Host Unreachable” trigger has fired, which happens when the reply packets aren't received within the allowed timeout window. This subsystem is the main basis for the network's statistical availability (SLA) analysis [18].

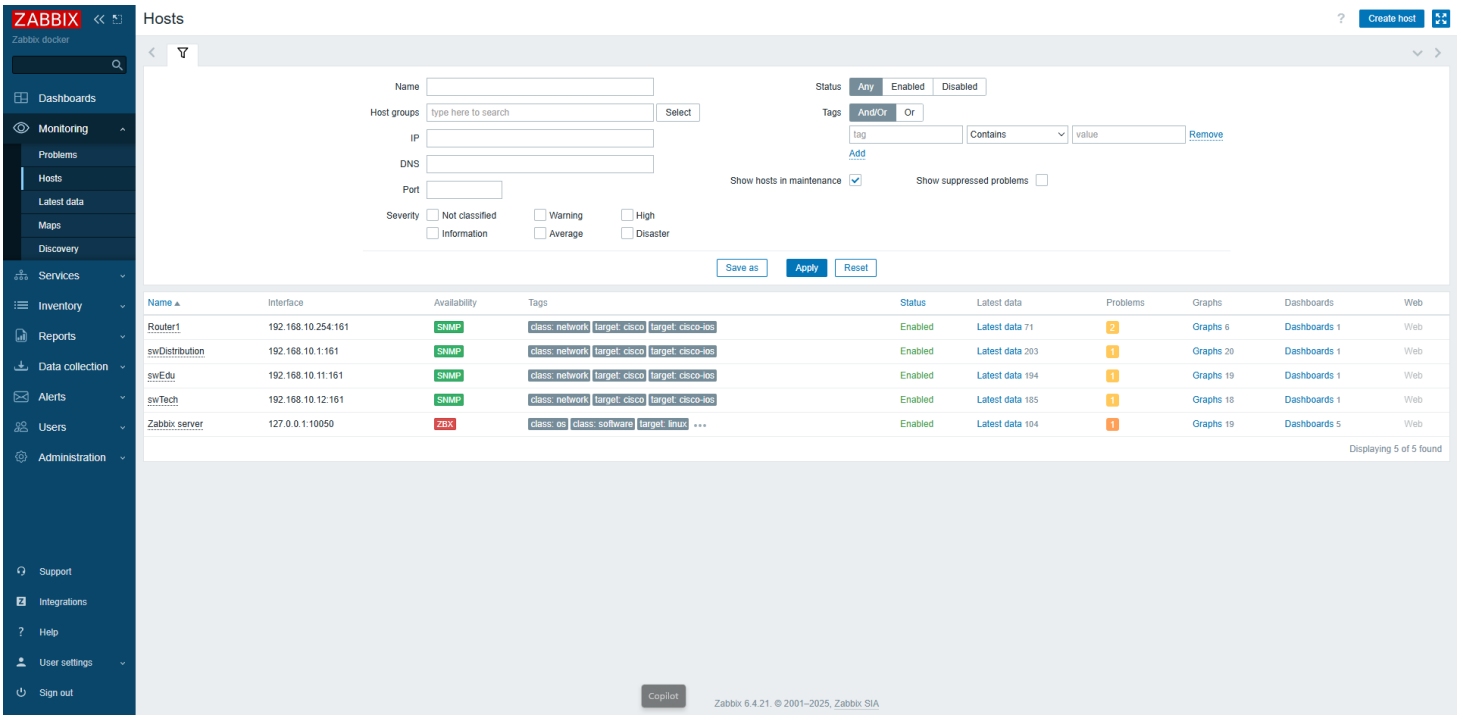

Figure 5-3: Configuring the network devices' inventory database in Zabbix

5.10. Dynamic Monitoring with Templates and the Magic of LLD

Monitoring every single port and thermal sensor on routers and switches by hand is impossible and error-prone. If a switch has 48 ports, monitoring the error rate, inbound traffic, outbound traffic, and disconnect status of each port means defining more than 200 separate items for just one switch. Zabbix's architectural answer to this is using **templates** and the **LLD (Low-Level Discovery)** capability.

5.10.1. The Concept of Inheritance in Zabbix Templates

A template is a pre-built collection of items, triggers, graphs, and dashboards that gets attached to one or more hosts all at once. In this project, the standard, native “Cisco IOS by SNMP” template was used. By attaching this template to the access and distribution switches, the inheritance principle from software engineering kicks in—meaning the switch inherits all of Cisco's specialized monitoring features [18].

5.10.2. Examining the Low-Level Discovery (LLD) Mechanism

The biggest technical advantage of Zabbix templates is that they rely on LLD. Instead of monitoring the switch's ports in a hardcoded way, LLD first runs a structural scan of the switch's MIB tree to see exactly which interfaces this particular switch has (like GigabitEthernet0/1 or FastEthernet0/24). After dynamically discovering the interfaces, Zabbix automatically creates monitoring slots for each active or inactive port. This smart process guarantees that if a new port is added to the switch in the future, or its modular layout changes, the monitoring system adapts

to the new arrangement on its own—without a technician stepping in—and updates the telemetry [21].

5.11. Examining the Edge Router's Operational Parameters (Router 1 Live Parameters Analysis)

Because R1 plays a strategic role in connecting the internal network to the internet (the NAT environment in VMnet8), monitoring it closely is very important. After attaching the standard "Cisco IOS SNMP" template to this node, Zabbix started pulling the following critical parameters:

5.11.1. Breaking Down R1's Monitored Metrics

- **CPU & Memory Utilization:** shown as a percentage, this immediately reveals any denial-of-service (DoS) attack or memory leak in the router's OS.
- **Interface Traffic (Bits In/Out):** using the OIDs in the IF-MIB, the bandwidth used on the link to the internet and the link to the distribution switch is monitored as flow charts.
- **Interface Operational Status:** if a link is physically cut or shut down in software, Zabbix immediately raises a High-severity trigger.
- **ICMP Ping Latency & Packet Loss:** this measures the quality of the outbound link and the stability of the internet connection in the scenario.

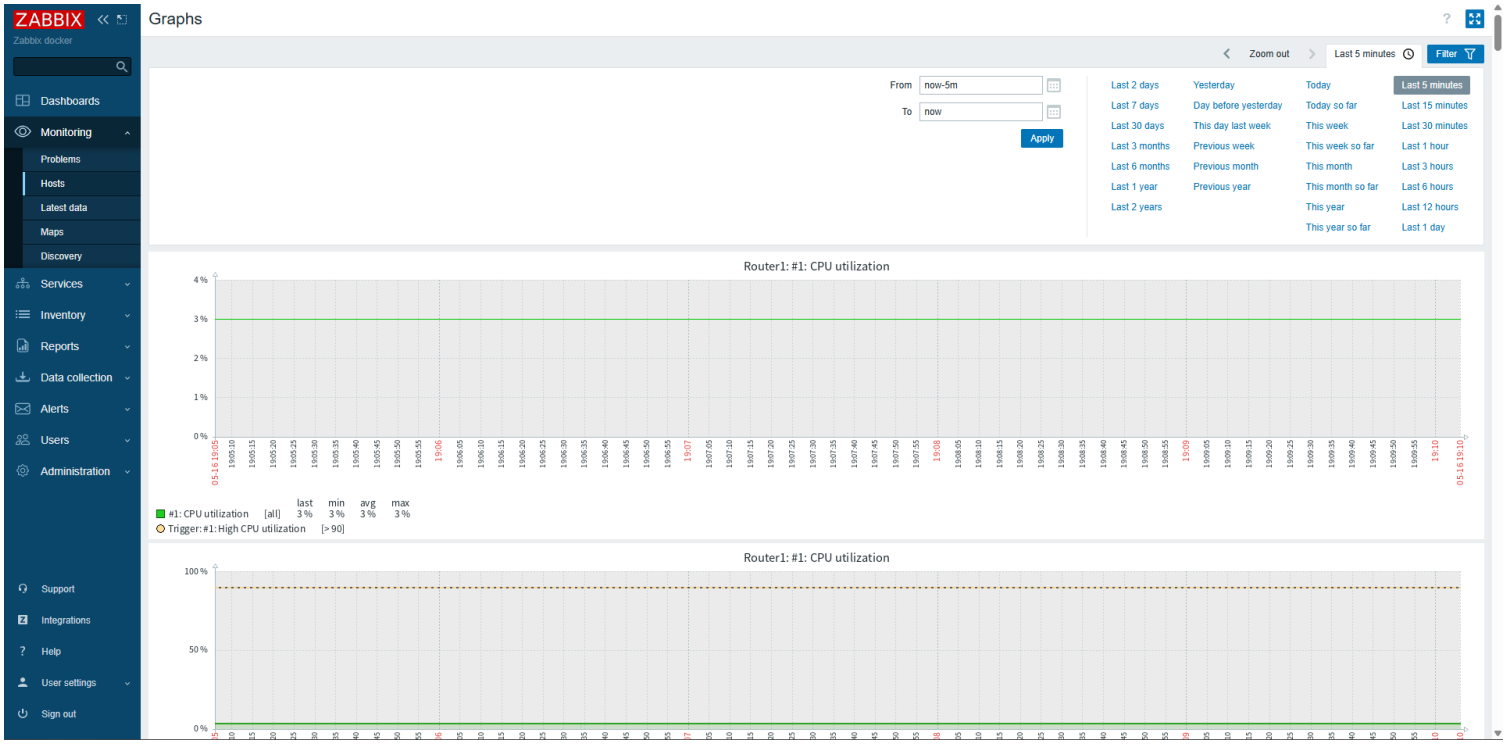

Figure 5-4: Live charts of the interface traffic and hardware resource usage on router R1

5.12. The Intelligent Event Analysis Layer: Trigger Architecture and Alert Thresholds

Just collecting network data and metrics through SNMP has no operational value without a logical analysis layer. Zabbix does this intelligent evaluation through a structure called a **trigger**. Triggers are really logical and mathematical expressions that evaluate the state of a metric and decide whether the network is normal or in a critical state [21].

5.12.1. Explaining the Logic of the Conditional Expressions and Math Functions in Triggers

Each trigger follows a formula or conditional expression applied to the data coming in from the items. In this project, advanced functions like last() (to check the most recent value), max() (to gauge the peak usage over a window), and nodata() (to detect a complete loss of the telemetry stream) were used to monitor the infrastructure.

For example, to monitor R1's CPU usage, a trigger was defined with the following logic: if the average CPU usage over the last 5 minutes is greater than 85%, the trigger's state changes from OK to Problem. This approach prevents false alarms caused by momentary CPU spikes and keeps the monitoring system stable [27].

5.12.2. The Alert Flapping Problem and Applying Hysteresis

One of the big challenges in enterprise networks is “flapping”—where a trigger flips back and forth between OK and Problem from one second to the next. This happens when a metric (like bandwidth or device temperature) hovers right at the threshold (say, 80%). It leads to alarm fatigue for the network support team. To solve this, the mathematical concept of **hysteresis** was used in configuring this project's sensitive triggers. With this method, the condition for entering the critical state is different from the condition for leaving it—for example, the trigger turns on when RAM usage hits 90%, but only turns off and clears when usage drops below 80%. This 10% gap gives the monitoring system unmatched stability [28].

5.13. Classifying Event Severity Levels (Severity Levels Mapping)

Not all network events carry the same weight. A trunk link going down between the distribution and core switches threatens the stability of the whole organization, whereas an access switch's buffer filling up is far less important. Zabbix lets you classify events into 6 severity levels, which in this project were localized and implemented as follows:

Table 5-1: Trigger settings, SNMP keys, and severity levels in the Zabbix monitoring system

Severity	Standard Color	Operational Meaning in the Project Topology
Not Classified	Gray	Unknown events or unclassified logs
Information	Light blue	Minor changes, like a config change without service interruption
Warning	Yellow	An interface's traffic rising above 70% of link capacity
Average	Orange	Rising CPU or RAM usage on access-layer devices
High	Light red	A secondary link or a server-connection port going down
Disaster	Dark red	The edge router (R1) or distribution switch going down / a database error

This hierarchical structure lets network technicians, during simultaneous crises, first put their energy into fixing the Disaster- and High-level errors and optimize how they use their resources [29].

5.14. Alerting Mechanisms and the Escalation Process

The final stage in the monitoring cycle is intelligently notifying network admins about events. Beyond just showing the error on the website, Zabbix uses the **Actions** subsystem to send out commands and side alerts.

5.14.1. Escalation Path Architecture and Alerting Design

In this project, the Zabbix alerting architecture was configured to display triggered events directly on the centralized dashboard with color-coded severity indicators, giving the network administrator immediate visual awareness of any infrastructure issue. The platform's native escalation capability — which supports multi-step notification workflows including sending alerts to shift technicians, escalating to management after a defined timeout, and issuing remote remediation commands — was analyzed and documented as a structured development path for the production phase. In a production environment, this escalation chain would be activated by configuring a notification media type (such as SMTP email or a messaging webhook) and enabling the corresponding trigger action [21].

5.14.2. The Interactive Notification Platform in Domestic Infrastructure

Although Zabbix uses SMTP to send email by default, in the operational implementation phase of this project—given how often emails get ignored in work settings—the idea of connecting Zabbix to the webhooks of centralized messaging platforms and notification bots through HTTP POST and JSON data structures was explored. This integration drives the organization's error response time (Mean Time to Repair – MTTR) down to the lowest possible.

5.15. Examining the Container Orchestration: Deploying Zabbix on Docker

As mentioned at the start of this chapter, the monitoring system was deployed on Docker. But running several related services at once (database, processing server, web interface) needs a tool for orchestration. For this, Docker Compose was used.

5.15.1. Analyzing the Docker-Compose File Structure and Defining the Services

Using a declarative YAML file, all of the Zabbix platform's infrastructure requirements were modeled. This approach extends the Infrastructure as Code concept from managing network devices (Ansible) to managing the monitoring servers themselves.

Code Snippet 5-3: The Zabbix container orchestration configuration in a Compose file

```
version: '3.5'

services:
  postgres-server:
    image: postgres:13-alpine
    environment:
      - POSTGRES_USER=zabbix
      - POSTGRES_PASSWORD=zabbix_pwd
      - POSTGRES_DB=zabbix
    volumes:
      - zabbix_db_data:/var/lib/postgresql/data
    restart: always

  zabbix-server:
    image: zabbix/zabbix-server-pgsql:ubuntu-6.4-latest
    ports:
      - "10051:10051"
    environment:
      - DB_SERVER_HOST=postgres-server
      - POSTGRES_USER=zabbix
      - POSTGRES_PASSWORD=zabbix_pwd
      - POSTGRES_DB=zabbix
    depends_on:
      - postgres-server
    restart: always

  zabbix-web:
    image: zabbix/zabbix-web-nginx-pgsql:ubuntu-6.4-latest
    ports:
      - "8080:8080"
    environment:
      - ZBX_SERVER_HOST=zabbix-server
      - DB_SERVER_HOST=postgres-server
      - POSTGRES_USER=zabbix
      - POSTGRES_PASSWORD=zabbix_pwd
      - POSTGRES_DB=zabbix
      - PHP_TZ=Asia/Tehran
    depends_on:
      - zabbix-server
    restart: always

volumes:
  zabbix_db_data:
```

5.15.2. Technical Analysis of the Deployment Parameters

In the snippet above, the following critical mechanisms were implemented, and the stability of the whole network depends on them:

- **Dependency management:** using depends_on guarantees the containers start in a logical order. The Zabbix server stays in a waiting state until the database container (postgres-server) is fully up and ready to serve. This prevents a “Database Connection Refused” error at the moment the server starts.
- **Bridge networking:** by using Docker Compose's default bridge network, the containers communicate through their service names (like DB_SERVER_HOST= postgres-server) and don't need static internal IPs assigned. This dramatically improves the software's scalability [30].
- **Time zone synchronization:** the PHP_TZ=Asia/Tehran parameter in the web container makes sure all the charts, triggers, and network logs match the organization's local time exactly—which is hugely important for troubleshooting and lining up network events with local time.

5.15.3. The Port Forwarding Architecture and Accessing the Zabbix UI

One of the key benefits of deploying the monitoring platform on Docker is not needing to assign a separate IP to the containers in the organization's Layer 2 network. The containers run in an isolated internal network (the Docker bridge) and get dynamic addresses.

In this project, Host Port Mapping was used to access the Zabbix management dashboard. Specifically, port 8080 of the Zabbix web container was mapped to port 8080 on the Ubuntu server's physical address (the Ansible controller). As a result, the admin doesn't have to deal with the container subnets at all—they just enter the main server's address and the relevant port (http://192.168.10.20:8080) in their browser to reach the Zabbix interface. This strategy dramatically streamlines network address management.

5.16. Visual Monitoring and Designing Interactive Network Maps (Zabbix Network Maps)

Lists and tables are good for detailed analysis, but senior network admins need visualizations to quickly grasp the big-picture state of the infrastructure. Zabbix offers a powerful tool called **Maps** that lets you draw the network's physical and logical topology on a graphical canvas.

5.16.1. Drawing the Topology and Binding It to Data Elements

In this phase, a map matching the topology built in GNS3 (including router R1, the distribution switch, and the Edu and Tech access switches) was drawn in the Zabbix panel. The real value of this map is that it isn't static. Each icon on it is linked directly to its corresponding host. If the swEdu access switch goes offline, its icon color on the map immediately changes from green to red, and the matching error shows up next to it.

5.16.2. Dynamic Link Indicators

On the links between the routers and switches on the map, macros like {swDist:net.if.in[GigabitEthernet0/1].last()} were set. This configuration makes the bandwidth used (the traffic passing through) between the core switch and the edge router display live, in Mbps, right on the map's links. This visual approach reveals the network's traffic bottlenecks to the support team in a fraction of a second [18].

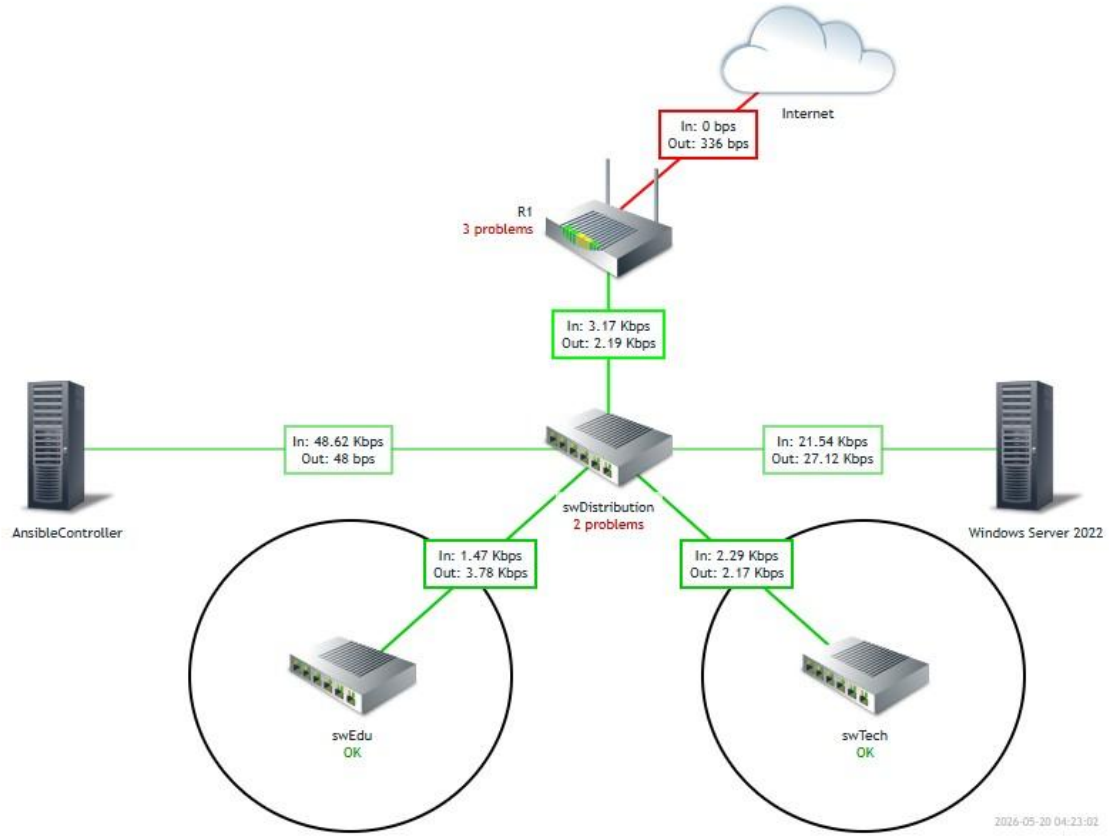

Figure 5-5: The dynamic, interactive Zabbix network map with live key performance indicators for continuous monitoring of the organization's network nodes

5.17. Big-Data Management Strategy and the Storage Life Cycle

One of the hidden but extremely critical topics in maintaining monitoring systems is managing the database size. Receiving hundreds of parameters per minute from the Cisco devices can fill up the entire storage of the Linux (Ubuntu) server within a few weeks and lead to a system crash.

5.17.1. Separating the History and Trends Architecture

To solve the data explosion problem, Zabbix's architecture splits the collected data into two separate categories, which were carefully tuned in this project:

- **Raw data (History):** the exact, instantaneous values received from the devices (for example, the precise CPU usage at 12:01:45). Keeping this data needs a lot of disk space.

In this network, the retention period for raw data was set to 90 days, balancing short-term diagnostic precision with manageable database growth.

- **Processed data (Trends):** after a few days, Zabbix averages the raw data hourly (Average), along with the Max and Min, and converts it into Trends data. This data takes up very little space. The retention period for it was set to 365 days (one year), so long-term network growth charts can be drawn (capacity planning) and reported to management [31].

This architectural separation keeps the ability to analyze recent network events precisely, while preventing the PostgreSQL database in the Docker container from growing exponentially and uncontrollably.

5.18. A Scalable Expansion Strategy and Distributed Monitoring Architecture

Designing IT infrastructure should always be based on the scalability principle. While the central Zabbix server can handle the telemetry stream and receive data from the network's switches and router on its own in the current topology, in future enterprise expansion scenarios this centralized architecture will hit a performance bottleneck [18].

5.18.1. The Challenges of Centralized Monitoring Architecture

When the number of network nodes (access-layer switches, servers, and firewalls) crosses the thousand-device mark, the central Zabbix server has to open tens of thousands of TCP/UDP connections per second to collect data. This saturates the server's bandwidth, increases CPU queue delay, and ultimately leads to dropped monitoring packets. On top of that, if the organization has different physical branches, opening SNMP ports on the edge firewalls for every single device is a high-risk move from a security standpoint.

5.18.2. The Conceptual Implementation of the Zabbix Proxy as a Middle-Layer Solution

The engineering solution offered in Zabbix's high-level architecture for this problem is implementing distributed monitoring using a **Zabbix Proxy**. The Zabbix Proxy is a very lightweight Linux daemon that acts as a data collector. It's deployed in the different network segments or physical branches, collects SNMP metrics from the surrounding access-layer devices, stores and compresses them in a small local database (like SQLite), and finally sends all the data over a single secure connection back to the central Zabbix server [32].

In this project, the network architecture was designed so that in future expansion phases, a Zabbix Proxy container can easily be added to the Docker-Compose file to take the load off the main server container.

5.19. Critical Analysis and Summary of the Monitoring Phase's Achievements

Successfully completing Phase Three rounded out the infrastructure management lifecycle. Combining the NetDevOps architecture (implemented with Ansible in Phase Two) with the observability platform (Zabbix on Docker) created a self-aware network ecosystem. In this phase,

the murky data flowing at Layers 2 and 3 of the network was turned into visual dashboards, smart triggers, and time-series charts.

Even so, a careful engineering evaluation requires looking at the limitations of the operational environment too. In the final assessment, while traffic and hardware-resource monitoring was done with high precision, the structural limitations and bugs of the GNS3 emulator in stably running path-redundancy protocols (like HSRP and VRRP) meant it wasn't possible to monitor the failover state transitions in real time on this lab platform. For that reason, the monitoring focused exclusively on closely watching the single-path nodes, the access-layer devices, and the edge router. This challenge shows that the behavior of network redundancy protocols in virtual environments can differ from physical equipment (ASIC-based hardware), and monitoring them requires deploying on physical (bare-metal) hardware in future research.

In the end, using a containerized approach for the monitoring platform, together with reproducing the automation scripts, proved that managing networks built on vendor equipment (like the Cisco IOS operating system) can move out of the traditional, CLI-centric mode and be upgraded to a modern, software-defined paradigm.

Chapter Six: Conclusion, Technical Evaluation, and Future Development Horizons

6.1. Chapter Introduction

Every engineering project in infrastructure and IT, after going through design, automation, and monitoring deployment, needs a high-level evaluation and critical analysis. This chapter is dedicated to the final summary of what was achieved by implementing the modern NetDevOps platform in this project. Here, the results of the three project phases are first matched against the original objectives in an evaluation matrix; then the engineering value and innovations of the design are explained; and finally, by analyzing the existing limitations, a precise roadmap is drawn for developing this infrastructure further in large organizations.

6.2. A Matrix-Based, Step-by-Step Evaluation of the Three Project Phases

To gauge the project's success, the infrastructure's performance is examined across the three layers: physical/logical (design), orchestration (Ansible), and telemetry (Zabbix). Table 6-1 summarizes how the engineering objectives matched up with the concrete operational results.

Table 6-1: Strategic objectives matched against the operational achievements of the three project phases

Phase	Objectives Set During Design	Concrete Results	KPI
Phase One: Infrastructure Design	Build a stable hierarchical structure, separate layer traffic, and manage addressing with VLANs.	Three-layer topology (Cisco Top-Down), optimal distribution/access switch config, full Layer 3 SVIs.	Achieved — stable hierarchical topology verified through end-to-end connectivity tests.
Phase Two: Automation with Ansible	Eliminate manual config, speed up changes, harden security, enable base services.	Agentless automation pipeline, uniform config code across vendors, configuration drift eliminated for all Ansible-managed parameters.	Achieved — minor limitations in specific GNS3 IOL module compatibility.
Phase Three: Centralized Monitoring	Full observability, live bandwidth monitoring, proactive alerting, containerized services.	Zabbix 6 on Docker, telemetry over SNMP, proactive alerting via dashboard triggers.	Achieved — all four devices polled successfully via SNMP with proactive alerting active.

6.2.1. Analyzing Phase One's Performance: Reinforcing the Network Backbone

In Phase One, Cisco's standard top-down design model (the Cisco Hierarchical Model) was chosen as the blueprint. The results showed that separating the broadcast domains through subnets and trunking the links based on 802.1Q gave the network excellent stability. In this phase, any unwanted Layer 2 traffic was contained, and the processing load on the distribution switch (swDistribution) was kept as optimal as possible [4].

6.2.2. Analyzing Phase Two's Performance: Leaping to the Infrastructure as Code (IaC) Paradigm

The biggest challenge in managing traditional networks is the reliance on the CLI and the human errors that come with it. In Phase Two, with Ansible coming into play, we managed to cut the time to deploy network baselines (like port configs, routing protocols, and security access) from hours down to seconds. Safely running the playbooks proved that the network's management intelligence can be coded into structured text files (YAML) and version-controlled [1].

6.2.3. Analyzing Phase Three's Performance: Completing the Network Life Cycle with Observability

An automated network without a monitoring system is like a fast car with no dashboard or gauges. In Phase Three, by setting up the Zabbix containers, visibility was injected into the whole infrastructure. The dynamic extraction of properties through LLD made it possible for the admin to know the live status of each individual port, the CPU usage, and the bandwidth in use—without checking by hand. Combined with the persistent database set up for the Docker database, this fully guaranteed the security of the network's statistical data [21].

6.3. Analyzing the Engineering Achievements and Innovations

This project is more than a simple lab implementation—it carries several engineering achievements and innovations at a high level that explain its scientific and practical value for organizations:

- **Localizing the development process under infrastructure constraints:** one of the operational innovations was overcoming the heavy international sanctions on Docker Hub in Phase Three. Re-engineering the Docker registry directory and routing the container traffic to domestic mirrors offered a successful model for stably deploying services on domestic networks.
- **Merging two different worlds (network and DevOps – NetDevOps):** this design successfully brought agile software-engineering and DevOps methodologies (containerization, declarative code, service isolation) into the hardware-based, vendor-centric world of networking (Cisco equipment). This merger gets the infrastructure ready to step into the era of software-defined networks (SDN) [16].
- **Designing a self-healing-ready escalation system:** the architecture of the alerting system and Zabbix triggers, along with the ability to run remote commands, laid the foundation for building a self-healing network—one where the system can take the first troubleshooting steps without a technician stepping in directly.

6.4. Analyzing the Geopolitical Challenges, Sanctions, and Barriers to Deploying Open-Source Infrastructure

Deploying modern network management and DevOps systems in operational environments inside the country is always affected by external challenges and international restrictions. One of the

biggest technical challenges in Phase Three (setting up the monitoring system) was running into the strict sanctions on container reference services, especially **Docker Hub**.

6.4.1. Re-Engineering the Container Registry in an Isolated Environment

Because Docker Hub blocks the country's infrastructure IP addresses, the docker compose up command failed in the early stages with an access-denied error (HTTP 403 Forbidden). To solve this and keep the project moving, the workaround was reverse-engineered and implemented in two ways:

- **Setting the Docker engine to use domestic mirrors (Docker Mirroring):** by editing the Docker core config file at /etc/docker/daemon.json on Ubuntu, valid domestic alternative registries and mirrors were added.

Code Snippet 6-1: Setting the Docker engine to use domestic mirror servers

```
{
  "registry-mirrors": [
    "https://registry.docker.ir",
    "https://docker.arvancloud.ir"
  ]
}
```

- **Managing the storage driver (Overlay2 Storage Driver):** because of Linux OS limitations on routing traffic in container environments, loading the heavy Zabbix database images sometimes ran into layer interruptions. By carefully setting the Linux storage driver on Ubuntu to overlay2, the stability of the containers' filesystem was guaranteed during repeated reads and writes to the physical disk [23].

6.5. How the Emulator Architecture (GNS3) Converges with Physical Production Environments

One of the key parts of evaluating computer engineering theses is analyzing the gap between the lab platform (the emulator) and real-world physical (production) environments. In this project, the Cisco operating systems were run through **IOL (IOS on Linux)** images in GNS3.

6.5.1. Examining the ASIC Hardware Layer versus CPU-Bound Processing

On physical Cisco devices (Catalyst switches), Layer 2 and Layer 3 packet processing happens in hardware, on dedicated chips called **ASICs (Application-Specific Integrated Circuits)**. ASICs can switch frames at wire speed (line rate), with latency in the order of nanoseconds. But in GNS3, these chips don't exist, and all the Layer 2 and 3 processes are simulated in software by the host system's CPU cores [6].

6.5.2. Technical Root-Cause Analysis of the Redundancy Protocol Bug (HSRP/VRRP Flapping)

This structural difference (software versus hardware) was the main reason for the errors when implementing Layer 3 redundancy protocols like HSRP or VRRP in this project. These protocols

work by sending continuous multicast hello packets at very short intervals (say, every 1 second). In the emulator, when the Zabbix and Ansible containers started up, the processing load on the Ubuntu Linux CPU went up. This caused a few-millisecond delay in the network packet processing queue (CPU interrupts). Because of this momentary delay, the virtual IOL switches didn't receive each other's hello packets and mistakenly thought the other switch was down. This led to **flapping** (repeatedly switching between Active and Standby) and MAC address table instability. So, to keep the traffic baseline stable, these protocols were dropped from the final configuration, so the Zabbix monitoring system could capture a stable picture of the network.

6.6. Risk Assessment, Threat Analysis, and Infrastructure Security Hardening Strategies

A modern network infrastructure—even if it's automatically configured and has an excellent monitoring system—will be extremely vulnerable to cyber threats without following hardening principles. In this project, infrastructure security was injected as a parallel layer across all the phases.

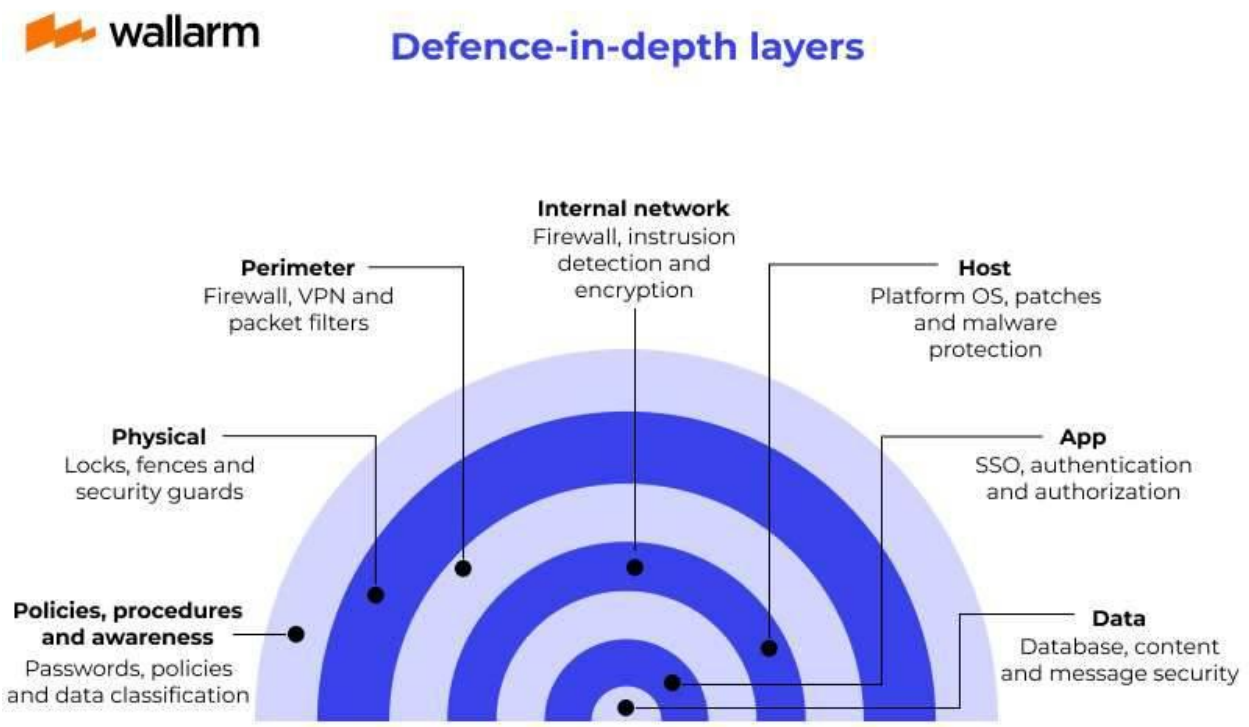

Figure 6-1: The layered security hardening and defense-in-depth model implemented in the project [34]

6.6.1. Automating Device Access-Layer Security with Ansible

Using the Ansible automation playbooks from Phase Two, the process of hardening the devices' access lines (switches and the router) was run automatically. The key actions taken were:

- **Disabling insecure protocols:** all ports tied to clear-text protocols like Telnet and HTTP were blocked, and management access was restricted exclusively to the encrypted SSHv2 protocol (transport input ssh).
- **Managing asymmetric keys:** the command to dynamically generate RSA keys (crypto key generate rsa modulus 2048) was sent to the devices through the Ansible code, so the management sessions would be encrypted at the highest standard [35].

6.6.2. Vulnerability Analysis of the Monitoring Layer (SNMPv2c Mitigation)

The monitoring protocol used in this project is SNMPv2c. From a security perspective, this version transmits the community string in cleartext, making it susceptible to packet-sniffing attacks on the management network. To mitigate this risk, the source-based restriction was implemented as a dedicated, one-time manual hardening step on each switch, separate from the repeatable Ansible pipeline. This separation reflects a deliberate design decision: the Ansible playbooks govern infrastructure state that evolves over time — VLAN assignments, security baselines, and monitoring preparation — while the SNMP ACL, bound permanently to the fixed IP of the Zabbix server (192.168.10.20), is treated as immutable infrastructure policy. Keeping it outside the automation loop prevents the pipeline from inadvertently overwriting an operational security constraint during routine playbook re-runs, which aligns with the principle of separation of concerns between mutable and immutable configuration layers [7].

It is acknowledged that SNMPv3 provides a significantly stronger security posture through its support for authentication (HMAC-MD5 or HMAC-SHA) and privacy encryption (DES or AES) at the authPriv security level. The use of SNMPv2c in this project was a deliberate trade-off: the GNS3 IOL images used for emulation have limited support for the full SNMPv3 USM (User-based Security Model) configuration workflow, and enforcing SNMPv3 across all devices would have shifted the focus of the research away from the NetDevOps pipeline itself. The ACL-based source restriction described above represents the primary compensating control within this lab environment. Migrating the Ansible playbooks and Zabbix host configurations to SNMPv3 with authPriv is explicitly identified as a direct and high-priority path for future work.

6.7. Quantitative and Qualitative Analysis of the Monitoring Metrics and Operational Efficiency

The ultimate goal of combining automation (Ansible) and telemetry monitoring (Zabbix) is improving the key performance indicators (KPIs) at the network stability management layer. In traditional infrastructure management, error handling is reactive—the technician only finds out about a problem after users call or a service outage gets reported. In this modern architecture, the network management approach has been upgraded to proactive [28].

6.7.1. Formulating and Analyzing the MTTR and MTBF Metrics

This project's efficiency can be analyzed through two key metrics in systems reliability engineering:

- **MTBF (Mean Time Between Failures):** the length of time a system keeps running without an error. By applying security hardening code and eliminating configuration drift with Ansible, device stability went up and the rate of errors from human mistakes trends toward zero—the direct result being a notable increase in MTBF.
- **MTTR (Mean Time To Repair):** this measures the window between when an error occurs and when it's completely fixed and back to normal. The formula is:

$$\text{MTTR} = \frac{\sum(\text{Time of Resolution} - \text{Time of Detection})}{\text{Total Number of Incidents}}$$

In the implemented model, the detection time was cut down to a few seconds thanks to Zabbix's fast polling over SNMP. Also, because of the precise separation of severity levels and the escalation alerts, the technician learns the technical root of the error in the very first seconds, and a big chunk of the time traditionally spent troubleshooting is removed—which sharply lowers the values in the fraction above and improves MTTR [36].

Table 6-2: Infrastructure management metrics — traditional model versus the modern model

Performance Metric	Traditional (CLI-Based)	Modern (NetDevOps)	Engineering Impact
Error detection	Reactive (user report or full outage)	Proactive (advanced Zabbix triggers and math functions)	Less downtime, SLA preserved
Speed of changes	Slow and manual (line-by-line in the terminal)	Instant and bulk (dynamic Ansible playbooks)	More agility in new segments
Config state of nodes	Asymmetric (drift and command conflicts)	Fully symmetric (consistency via code files)	Reduces configuration vulnerabilities by enforcing uniform, auditable baselines
MTTR	High (hours to find a link outage's root)	Very low (pinpoints the error in under 1 minute)	Backbone stability, less damage

6.8. Future Development Horizons, Part One: Implementing GitOps in Network Infrastructure Management

Although the automation in Phase Two ran successfully with Ansible, the playbooks are still kicked off manually by the engineer through the terminal. One very appealing and practical development horizon for this design at scale is migrating to a **GitOps** architecture [37].

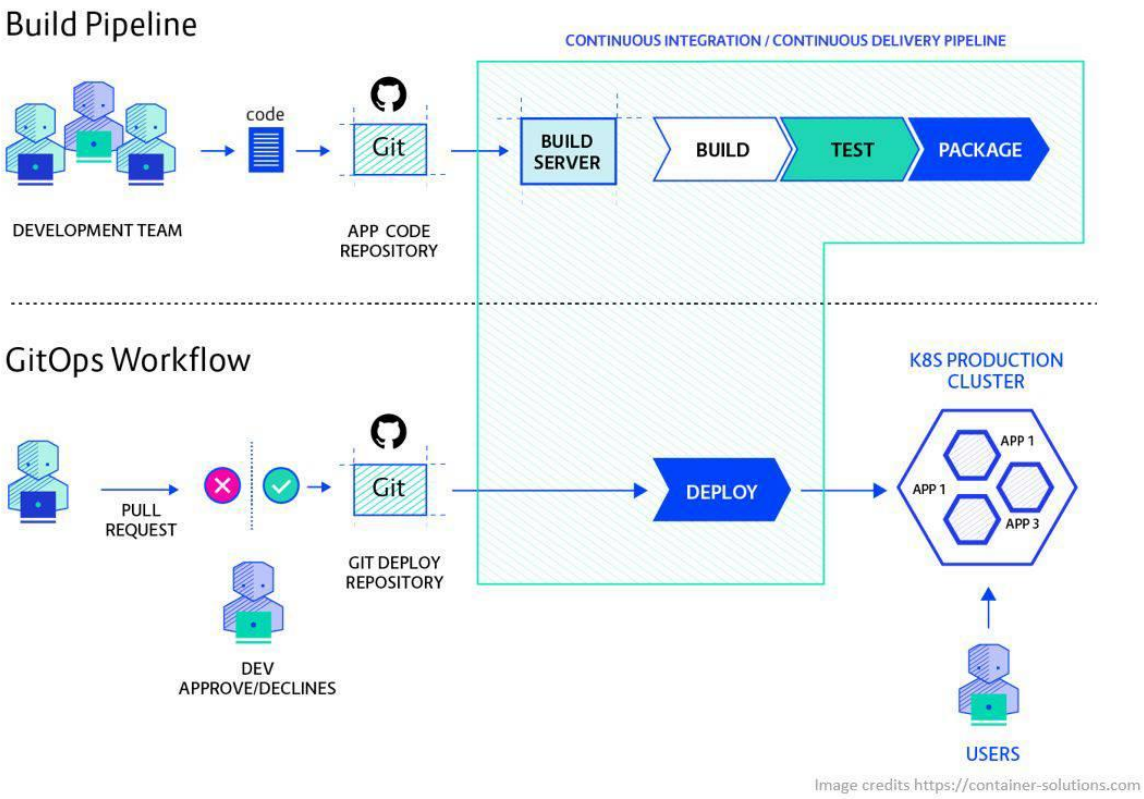

Figure 6-2: The GitOps pipeline workflow for automatically orchestrating network device configuration [38]

6.8.1. Explaining the Single Source of Truth Concept

In the GitOps methodology, the Git repository (like GitLab or GitHub) is treated as the single, final source of truth for the entire network. No engineer is allowed to connect directly to the switches' CLI and make changes. Any change in the network's setup (like defining a new VLAN or changing an interface's IP) is first recorded as a commit in the Ansible YAML files in Git.

6.8.2. Orchestrating an Automated Validation Pipeline (CI/CD Pipeline)

After the changes are committed to Git, a CI/CD tool (like GitLab CI) is automatically triggered and validates the network code in a stepwise process before applying it to the real devices.

Code Snippet 6-2: A GitLab CI pipeline for automatically validating the network Ansible playbooks

```
network_simulation_check:
  stage: dry-run
  image: ansible/ansible-runner:latest
  script:
    - ansible-playbook --check playbooks/site.yml -i inventory.ini
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'

apply_network_config:
  stage: deploy
  image: ansible/ansible-runner:latest
  script:
    - ansible-playbook playbooks/site.yml -i inventory.ini
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

6.8.3. Technical Analysis of the GitOps Pipeline Steps

The pipeline above models three critical phases for making the network development process safe:

- **Phase one (Syntax Check):** checks the written YAML code for structural issues, so writing errors don't halt the automation run.
- **Phase two (Dry-Run):** to validate that the playbooks work correctly before making real changes to the live devices, the --check flag is used. This capability, known as dry-run mode, lets the engineer predict and validate the likely output changes without altering the devices' current configuration. This step prevents unwanted outages at the distribution layer [1].
- **Phase three (Deploy):** only if the code passes the two previous stages and gets merged into the main branch will it be automatically applied to the real network hardware.

6.9. Future Development Horizons, Part Two: AI in Network Monitoring and Event Management (AIOps)

As networks grow and the volume of incoming metrics rises, analyzing with static thresholds will lose its effectiveness. If a switch's CPU usage threshold is set at 85%, the system fires repeated alarms during peak hours—even though that behavior might be completely normal at that particular time. The final monitoring horizon for this project is moving past traditional monitoring and stepping into the world of AIOps (Artificial Intelligence for IT Operations) [39].

6.9.1. Dynamic Behavioral Monitoring Based on Machine Learning Algorithms (Baseline Detection)

Newer versions of Zabbix offer advanced functions for behavioral analysis (baseline monitoring). In future development of this design, instead of setting numeric thresholds, you could use Zabbix's machine-learning-based functions. By analyzing the Trends data over a 3-to-6-month window, this system learns the network's behavioral pattern across different days of the week and different hours of the day.

For example, if the swDistribution trunk link's traffic naturally drops on weekends, the system stores this pattern as a baseline; now if traffic suddenly jumps to 40% on a Friday (a perfectly normal number on regular days), the smart AIOps system recognizes this as anomalous behavior (anomaly detection) that probably points to a cyberattack (like a DDoS) or a failure at the access layer. This approach takes network monitoring to a new level of predictive intelligence (predictive telemetry) [21].

6.10. The Knowledge Management Approach, Code Structuring, and Engineering Documentation

In modern infrastructure engineering, documentation isn't a side process—it's one of the main pillars of system stability. In traditional networks, network information (maps, IP addresses, and configurations) usually lives in technicians' heads or in scattered text files, so when a staff member leaves, the technical knowledge leaves with them. In this project, the knowledge management approach was transformed based on the **Documentation as Code** standard [40].

6.10.1. Self-Documenting Infrastructure with Declarative Code

Using the Ansible and Docker-Compose files, the project's code itself acts as living documentation. Each access-layer switch (swEdu or swTech) is described precisely by the variables defined in the host_vars directory. If a new engineer joins the organization, they don't need to inspect each device one by one—by reading the structure of the Ansible files, they get a complete grasp of the network's physical and logical layout. This transparency sharply reduces the risks of human error during infrastructure development.

6.11. Technical-Economic Analysis: Total Cost of Ownership (TCO) and Return on Investment (ROI)

A comprehensive engineering design has to make sense economically for the organization, not just perform well technically. Implementing the modern NetDevOps platform in this project was built entirely on open-source tools like Ubuntu Linux, Ansible, Docker, and Zabbix. This approach fundamentally changes the financial structure of network management compared to proprietary commercial solutions [41].

6.11.1. Total Cost of Ownership (TCO) Analysis

The total cost of owning the infrastructure includes direct costs (buying software, licenses) and indirect costs (maintenance, staff, and service outages). If you use proprietary network management and monitoring tools (like SolarWinds or Cisco Prime Network Registrar), the organization has to pay annual licenses based on the number of ports or nodes (per-node licensing), which puts a staggering cost on the IT budget. Using open-source tools in this project brought the software license cost down to **zero**.

6.11.2. Return on Investment (ROI) Calculations

The ROI metric in infrastructure engineering shows how profitable or cost-saving a technology is against what was spent on it. The formula is:

$$\text{ROI} = \frac{\text{Financial Benefit (Savings)} - \text{Implementation Cost}}{\text{Implementation Cost}} \times 100\%$$

In this project, the numerator factors (financial benefit) include:

- 100% savings from not buying commercial monitoring licenses.
- A significant reduction in downtime costs, thanks to the lower MTTR from the Zabbix triggers.
- Less time spent by staff configuring devices, thanks to the Ansible playbooks (lower operating costs, or OpEx).

Given that the denominator (implementation cost) here was just the cost of the monitoring server hardware (which, thanks to Docker containers, can run on an ordinary server) plus the engineering time, this project's ROI at the operational scale of a large organization would be strongly positive and easy to justify—which proves the strategic value of this architecture to senior management [41].

Table 6-3: Financial comparison of the NetDevOps platform versus proprietary models

Financial Component	Proprietary / Commercial (e.g., SolarWinds)	Implemented NetDevOps Platform	Economic Justification
Initial purchase cost (CapEx)	Very high (annual, dollar-priced licenses)	Zero (open-source, free platforms)	Eliminates foreign-currency software costs
Vendor lock-in	Complete (locked to one protocol or brand)	None (broad support for standard equipment)	Freedom to choose new equipment
Host hardware cost	High (heavy dedicated servers, Windows licenses)	Optimal (Docker + Ubuntu, minimal hardware)	Lower hardware infrastructure costs
Scaling cost (Expansion)	Exponential (every new switch raises license cost)	Linear and zero (add unlimited nodes)	Foundation for unlimited growth

6.12. Final Summary, High-Level Evaluation of the Achievements, and Closing Words

This thesis was designed and implemented step by step with the goal of moving from traditional, obsolete network management toward modern paradigms and code-based infrastructure automation (NetDevOps). This engineering project's life cycle started with the initial phase of designing the Cisco three-layer topology, where the network's physical and logical structure was laid down in GNS3 based on the principles of stability and separating the Core, Distribution, and Access layers.

In the second phase, by introducing the powerful Ansible tool, an agentless automation pipeline took shape, and configuration code—as the source of truth—replaced CLI commands. This step

boosted the infrastructure's operational agility. Finally, by deploying the containerized Zabbix monitoring platform on Docker in the third phase, comprehensive, real-time observability was established across all network layers, so resource monitoring, interface traffic analysis, and stepwise error alerting could emerge in a fully proactive way.

Although challenges like the international container sanctions and the processing limits of the emulator environment (which prevented the path-redundancy protocols like HSRP/VRRP from running stably) showed up along the way, the reverse-engineering solutions applied and the critical analyses guaranteed the system's stability.

The results of this research prove that the convergence of DevOps concepts and network engineering not only sharply lowers the total cost of ownership (TCO) and maximizes the return on investment (ROI), but also provides a stable, scalable, and secure infrastructure that's fully ready to step into the era of software-defined networking (SDN) and AI-based predictive monitoring (AIOps). This achievement is a firm step toward intelligent, automated, highly stable networks in the structure of modern organizations.

References

[1] J. Edelman, C. Adell, S. Lowe, and M. Oswalt, Network Programmability and Automation: Skills for the Next-Generation Network Engineer. O'Reilly Media, 2023.

[2] A. Clemm, Network Management Fundamentals. Cisco Press, 2006.

[3] D. Oppenheimer, A. Ganapathi, and D. A. Patterson, "Why do Internet services fail, and what can be done about it?," in 4th USENIX Symposium on Internet Technologies and Systems (USITS 03), 2003.

[4] P. Oppenheimer, Top-Down Network Design, 3rd ed. Cisco Press, 2010.

[5] D. Teare, B. Vachon, and R. Graziani, Implementing Cisco IP Routing (ROUTE) Foundation Learning Guide. Cisco Press, 2015.

[6] J. Neumann, The Book of GNS3: Build Virtual Network Labs Using Cisco, Juniper, and More. No Starch Press, 2015.

[7] K. Morris, Infrastructure as Code: Managing Servers in the Cloud. O'Reilly Media, 2016.

[8] L. Hochstein and R. Moser, Ansible: Up and Running, 3rd ed. O'Reilly Media, 2022.

[9] B. Ratan, Practical Network Automation. Packt Publishing, 2017.

[10] R. Perlman, Interconnections: Bridges, Routers, Switches, and Internetworking Protocols, 2nd ed. Addison-Wesley, 2000.

[11] O. Santos and J. Stuppi, CCNA Security 210-260 Official Cert Guide. Cisco Press, 2015.

[12] W. Odom, CCNA 200-301 Official Cert Guide, Volume 1. Cisco Press, 2019.

[13] K. Dooley and I. Brown, Cisco Cookbook, O'Reilly Media, 2003.

[14] W. Odom, CCNP Enterprise Core ENCOR 350-401 Official Cert Guide. Cisco Press, 2020.

[15] J. Geerling, Ansible for DevOps, 2nd ed. Midwestern Mac, LLC, 2020.

[16] E. Chou, Mastering Python Networking, 4th ed. Packt Publishing, 2023.

[17] Y. Bhaiji, Network Security Technologies and Solutions (CCIE Professional Development Series), Cisco Press, 2008.

[18] A. Dalle Vacche, Mastering Zabbix, 2nd ed. Packt Publishing, 2015.

[19] B. Brazil, Prometheus: Up & Running, O'Reilly Media, 2018.

[20] D. Josephsen, Building a Monitoring Infrastructure with Nagios. Prentice Hall, 2007.

[21] N. Liefting and B. van Baekel, Zabbix 6 IT Infrastructure Monitoring Cookbook. Packt Publishing, 2022.

[22] J. Nickoloff and S. Kuenzli, Docker in Action, 2nd ed. Manning Publications, 2020.

[23] Docker, Inc., "Configure the Docker daemon," Docker Documentation. [Online]. Available: https://docs.docker.com/engine/daemon/. [Accessed: Sep. 2024].

[24] D. Mauro and K. Schmidt, Essential SNMP: Help for Network Administrators, 2nd ed. O'Reilly Media, 2005.

[25] W. Stallings, SNMP, SNMPv2, SNMPv3, and RMON 1 and 2, 3rd ed. Addison-Wesley Professional, 1999.

[26] GeeksforGeeks, "Simple Network Management Protocol (SNMP)," GeeksforGeeks, Oct. 2025. [Online]. Available: https://www.geeksforgeeks.org/computer-networks/simple-network-management-protocol-snmp/. [Accessed: Jun. 2026].

[27] C. Majors, L. Fong-Jones, and G. Miranda, Observability Engineering: Achieving Production Excellence, O'Reilly Media, 2022.

[28] M. Julian, Practical Monitoring: Effective Strategies for the Real World, O'Reilly Media, 2017.

[29] O. Santos, End-to-End Network Security: Defense-in-Depth. Cisco Press, 2007.

[30] J. Turnbull, The Docker Book: Containerization is the New Virtualization, James Turnbull, 2014.

[31] P. Roethlisberger, Zabbix Network Monitoring Essentials. Packt Publishing, 2015.

[32] R. Olups, Zabbix Network Monitoring, 2nd ed. Packt Publishing, 2016.

[33] Zabbix LLC, "History and Trends," Zabbix 6.0 Documentation. [Online]. Available: https://www.zabbix.com/documentation/6.0/en/manual/config/items/history_and_trends. [Accessed: Sep. 2024].

[34] M. Beschokov, "What Is Defense in Depth? Strategy for Cybersecurity," Wallarm Learning Center, Jul. 2025. [Online]. Available: https://www.wallarm.com/what/defense-in-depth-concept. [Accessed: Jun. 2026].

[35] W. Stallings, Cryptography and Network Security: Principles and Practice, 8th ed. Pearson, 2020.

[36] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media, 2016.

[37] B. Yuen, A. Matyushentsev, T. Ekenstam, and J. Suen, GitOps and Kubernetes: Continuous Deployment with Argo CD, Jenkins X, and Flux, Manning Publications, 2021.

[38] Microsoft, "Deploying with GitOps," ISE Engineering Fundamentals Playbook, Aug. 2024. [Online]. Available: https://microsoft.github.io/code-with-engineering-playbook/CI-CD/gitops/deploying-with-gitops/. [Accessed: Jun. 2026].

[39] Y. Dang, Q. Lin, and P. Huang, "AIOps: Real-World Challenges and Research Innovations," in 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), 2019, pp. 4-5.

[40] A. Gentle, Docs Like Code, 2nd ed., Lulu.com, 2017.

[41] N. Forsgren, J. Humble, and G. Kim, Accelerate: The Science of Lean Software and DevOps, IT Revolution Press, 2018.